

Technical Handbook

Version 1.1 · June 4, 2026

Autor	Maximilian Jakob Maag B.Sc.
Audience	Administrators, Developers, Project Managers
System	Infrastructure Webshop
Stack	Go · PostgreSQL · HTMX · OpenTofu · GitLab · Linode
Scope	Architecture · Operations · Development · Product Lifecycle

Contents

I	Overview & Architecture	6
1	Introduction	7
1.1	Purpose of This Handbook	7
1.2	Target Audiences	7
1.3	Conventions Used in This Handbook	8
1.4	System At a Glance	8
2	C4 Architecture Model	9
2.1	Level 1 — System Context	9
2.1.1	Actors	9
2.1.2	External Systems	10
2.2	Level 2 — Container Diagram	11
2.3	Level 3 — Component Diagram	11
2.3.1	Component Descriptions	12
2.4	Level 4 — Key Code Flows	13
2.4.1	Order Placement Flow (Project Manager)	13
2.4.2	Order Approval & Provisioning Flow	14
2.4.3	Request-Response Lifecycle (HTMX)	14
3	Design Decisions & Technology Rationale	15
3.1	Go as Application Language	15
3.2	Server-Side Rendering with HTMX	15
3.3	templ for Type-Safe HTML Components	16
3.4	Tailwind CSS without DaisyUI	16
3.5	No Alpine.js	17
3.6	PostgreSQL with pgx (No ORM)	18
3.7	Single-Container Architecture	18
3.8	OpenTofu over Terraform	18
3.9	GitLab Managed Terraform State	18
3.10	Per-Resource Isolated State	19
3.11	Webhook-Driven Pipeline Trigger	19
3.12	Internationalization: 25 Languages	19
3.13	Session Security: Encrypted Cookies	19
3.14	Password Hashing: bcrypt	20

3.15	Static Asset Embedding: <code>embed.FS</code>	20
II	Infrastructure as Code	22
4	GitLab Repository Concept	23
4.1	Overview	23
4.2	Repository File Structure	23
4.3	State Management	24
4.4	CI/CD Pipeline Stages	24
4.5	Webhook Integration with the Webshop	27
4.6	Dependency Between Modules	28
5	OpenTofu Modules	29
5.1	network-core	29
5.2	vm-platform	29
5.3	k8s-platform	30
5.4	shared-services	30
5.5	observability	30
III	Administrator Guide	31
6	Initial Setup & Installation	32
6.1	Prerequisites	32
6.2	Environment Variables	32
6.3	Deployment on a Docker Host	33
6.4	Deployment on Kubernetes	35
6.5	Database Migrations	41
6.6	First Login	41
7	System Configuration	42
7.1	GitLab Sources	42
7.2	Deployment Environments	42
7.3	SMTP Configuration	42
7.4	AI Translation	43
7.5	Branding	43
7.6	Currencies	43
8	User Management	44
8.1	Roles	44
8.2	Creating a Local Account	44
8.3	Editing Users	44
8.4	Deactivating Users	45
9	Audit Log	46
9.1	Logged Events	46
9.2	Filtering	46

9.3	Export	47
IV	Developer Guide	48
10	Development Environment	49
10.1	Prerequisites	49
10.2	Quick Start	49
10.3	Makefile Targets	50
11	Codebase Structure	51
11.1	Directory Layout	51
11.2	Layered Architecture	52
11.3	Template System (templ)	52
11.4	Design System	53
11.5	Adding Translations	57
11.6	Database Migrations	58
12	Adding a New Feature	59
12.1	Example: Adding a New Admin Page	59
12.1.1	Step 1: Create the Migration	59
12.1.2	Step 2: Add the Model	59
12.1.3	Step 3: Define the Repository Interface	59
12.1.4	Step 4: Implement the Repository	60
12.1.5	Step 5: Add the Handler	60
12.1.6	Step 6: Create the templ Component	61
12.1.7	Step 7: Add i18n Keys	61
12.1.8	Step 8: Wire Up in main.go	61
12.1.9	Step 9: Test	61
V	Project Manager & Product Lifecycle Guide	62
13	Ordering Infrastructure	63
13.1	The Order Lifecycle	63
13.2	Placing an Order	63
13.3	Tracking Status	63
13.4	Using an Existing Deployment as a Template	64
13.5	Decommissioning	64
14	Introducing a New Product	65
14.1	Overview	65
14.2	Step 1: Plan the Product	65
14.3	Step 2: Prepare the OpenTofu Module	66
14.4	Step 3: Configure the GitLab Webhook	66
14.5	Step 4: Create a Category (if needed)	67
14.6	Step 5: Create the Product	67

14.6.1 Basic Information	67
14.6.2 Parameters	67
14.6.3 Deployment Environments	68
14.6.4 Multiple Webhooks (Pipeline Arrays)	68
14.7 Step 6: Configure AI Translation	68
14.8 Step 7: Global and Category Parameter Sets	69
14.9 Step 8: Test the Full Order Flow	69
14.10 Step 9: Test with a Project Manager Account	70
14.11 Step 10: Product is Live	70
15 Project Management	71
15.1 Projects	71
15.2 Infrastructure Overview	71
15.3 Cost Center Assignment	71
A Supported Languages	73
B Environment Variable Reference	74
C OpenTofu Module Variable Reference	75
C.1 network-core Variables	75
C.2 vm-platform Variables	75
C.3 k8s-platform Variables	76
C.4 shared-services Variables	76
C.5 observability Variables	76
D HTTP Route Reference	78
E Requirements Traceability Matrix	79
F Compiling This Handbook	81

List of Figures

2.1	C4 Level 1 — System Context Diagram	9
2.2	C4 Level 2 — Container Diagram	11
2.3	C4 Level 3 — Component Diagram (Webshop Application)	12
4.1	Module dependency graph	28
6.1	Docker Host deployment architecture	33
6.2	Kubernetes deployment architecture	36
13.1	Order lifecycle state machine	63

List of Tables

1.1	Technology stack overview	8
2.1	Component descriptions	13
4.1	GitLab repository overview	23
4.2	State name convention per module	24
6.1	Required environment variables	33
6.2	Key Helm values	37
8.1	User roles and capabilities	44
9.1	Audited actions	46
10.1	Makefile targets	50
A.1	Supported language codes	73
B.1	Environment variable reference	74
C.1	network-core variables	75
C.2	vm-platform variables	76
C.3	k8s-platform variables	76
C.4	shared-services variables	76
C.5	observability variables	77
D.1	HTTP route reference (selected)	78
E.1	Requirements traceability matrix	80

Listings

4.1	.gitlab-ci.yml for vm-platform (webhook-triggered module)	24
4.2	.gitlab-ci.yml for singleton modules (manual apply)	26
6.1	infra/docker-host/nginx.conf	34
6.2	Helm install with required values	36
6.3	Example secrets.values.yaml (do not commit)	36
6.4	Kubernetes Secret for the webshop	38
6.5	Migration Job	38
6.6	Deployment and Service	38
6.7	Ingress with cert-manager TLS	39
6.8	HorizontalPodAutoscaler	40

Part I

Overview & Architecture

Chapter 1

Introduction

1.1 Purpose of This Handbook

The PORR AG Infrastructure Webshop is a self-service portal that enables Digital Unit (DU) Admins and Project Managers to order, manage, and decommission IT infrastructure on Linode/Akamai Cloud through a browser-based interface. This handbook is the authoritative reference for everyone who operates, extends, or uses the system.

It covers:

- The complete system architecture described with the C4 model
- Infrastructure-as-Code modules and the GitLab CI/CD concept
- Architecture and design decisions with their rationale
- Step-by-step administrator operations
- Developer onboarding, code structure, and extension patterns
- The full product lifecycle from creation to decommissioning

1.2 Target Audiences

DU Admin	Responsible for approving orders, monitoring all infrastructure, and ordering directly without an approval step. Reads Part III and Part V .
Webshop Admin	Responsible for the product catalog, system configuration, and local user accounts. Reads Part III entirely.
Developer	Extends or maintains the webshop codebase and the OpenTofu modules. Reads Part I and Part IV .
Project Manager	Places infrastructure orders and manages projects. Reads Part V .

1.3 Conventions Used in This Handbook

monospace File paths, commands, code, environment variables, and route patterns.

italic Technical terms on first use.

bold UI labels, button names, and menu items.

Note boxes Yellow-highlighted boxes providing additional context or important background information.

Important boxes Orange-highlighted boxes warning about actions that can cause data loss or service disruption.

Tip boxes Green-highlighted boxes with shortcuts and best practices.

1.4 System At a Glance

The webshop is a single Go binary that renders HTML server-side and delivers page fragments via HTMX for a responsive, app-like feel. Orders are fulfilled by triggering GitLab CI/CD pipelines that run OpenTofu (a Terraform-compatible open-source tool) to provision or destroy cloud resources on Linode/Akamai. The system is stateless above the PostgreSQL layer, enabling horizontal scaling without coordination.

Table 1.1: Technology stack overview

Layer	Technology	Version
Backend	Go	1.22+
Templates	templ	0.3+
Frontend	HTMX	2.0
CSS	Tailwind CSS	3.x
Database	PostgreSQL	15+
DB Driver	pgx/v5 (jackc)	5.x
IaC	OpenTofu	1.9+
Cloud	Linode/Akamai	—
CI/CD	GitLab CE	16+
State Backend	GitLab Managed TF State	—
Auth	OIDC (Entra ID) + local	—

Chapter 2

C4 Architecture Model

The system is described using the *C4 model* (Context, Container, Component, Code). Each level zooms in one step deeper.

2.1 Level 1 — System Context

Figure 2.1 shows who uses the webshop and which external systems it depends on.

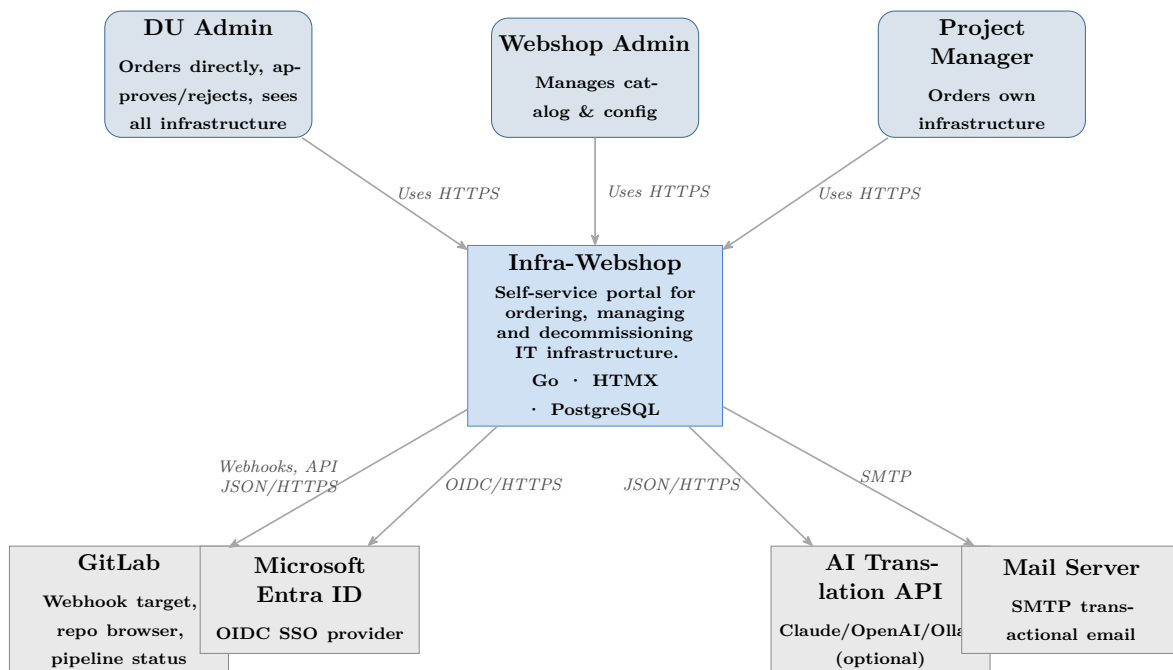


Figure 2.1: C4 Level 1 — System Context Diagram

2.1.1 Actors

DU Admin (Digital Unit Administrator) Authenticates via Microsoft Entra ID SSO. Has the system role `admin`. Can order infrastructure directly (no approval

required), approve or reject orders from Project Managers, view all projects and infrastructure, and decommission any resource.

Webshop Admin Uses a local account (no SSO). Has the system role `shop_admin`. Manages the product catalog, system configuration (SMTP, AI, branding, currencies), environments, GitLab sources, cost centers, and local users. Cannot place orders.

Project Manager Authenticates via Entra ID SSO. Has the system role `user`. Places orders (which require DU Admin approval), manages their own projects, and can decommission their own infrastructure.

2.1.2 External Systems

GitLab One or more self-hosted or cloud GitLab instances. The webshop connects to GitLab for three purposes: (1) browsing repositories and importing `variables.tf` parameter definitions, (2) triggering provisioning and decommissioning webhooks, and (3) polling pipeline status to update order state.

Microsoft Entra ID The OIDC identity provider for DU Admins and Project Managers. The webshop implements the Authorization Code Flow. Local accounts for Webshop Admins bypass this provider entirely.

AI Translation API Optional. Used to auto-translate product names and descriptions into all 25 supported languages. Multiple backends are supported: cloud-based (Claude, OpenAI, Azure OpenAI) and on-premise (Ollama, LocalAI). The feature is hidden when no provider is configured.

Mail Server Any SMTP server. Sends order confirmations, approval request notifications, approval/rejection notices, deployment completion alerts, and failure notifications.

2.2 Level 2 — Container Diagram

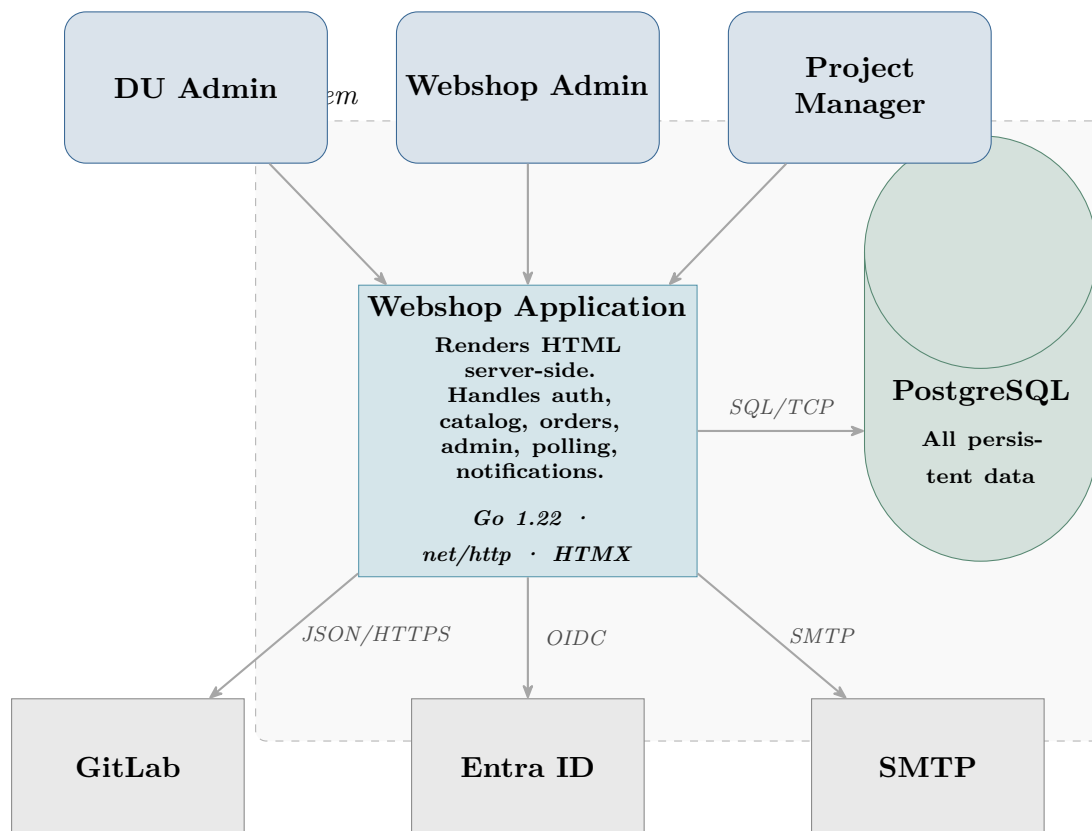


Figure 2.2: C4 Level 2 — Container Diagram

The system has exactly two containers:

Webshop Application A single stateless Go binary. It handles HTTP requests, renders HTML via `templ` components, serves HTMX fragments, coordinates all business logic, runs the GitLab polling goroutines, and manages outbound SMTP. Being stateless above the database layer means multiple replicas can run behind a load balancer without any coordination layer.

PostgreSQL Database The single source of truth. Stores the product catalog, orders, infrastructure state, audit log, branding, configuration, users, and exchange rates. The database is the only stateful component.

2.3 Level 3 — Component Diagram

Figure 2.3 decomposes the Go application container into its major logical components.

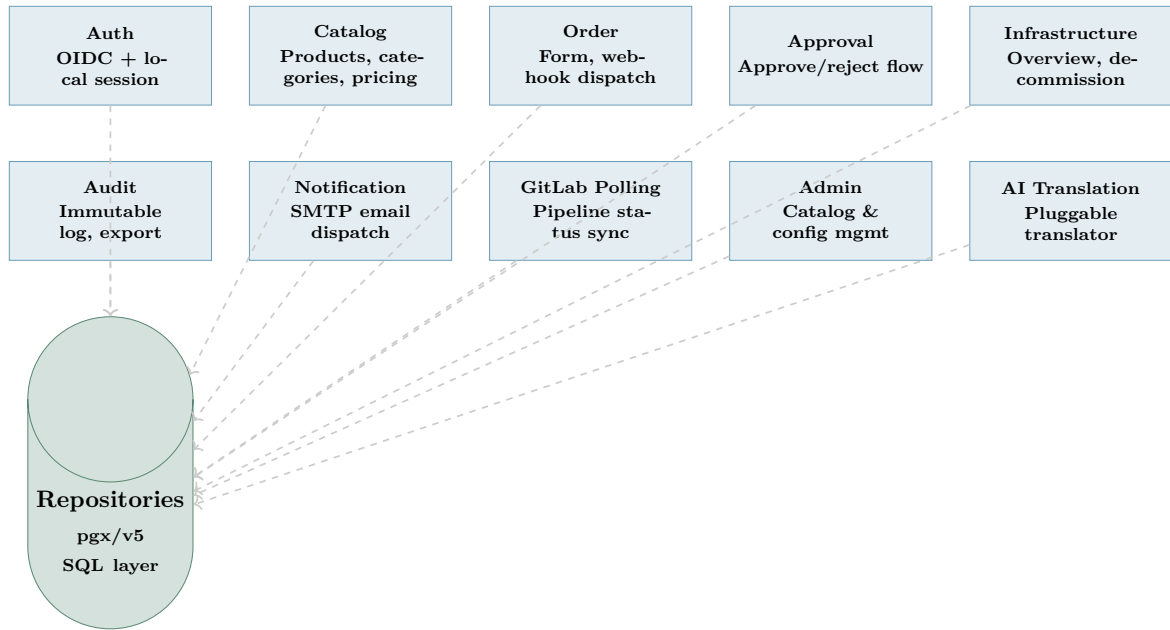


Figure 2.3: C4 Level 3 — Component Diagram (Webshop Application)

2.3.1 Component Descriptions

Component	Responsibility
Auth	Handles OIDC Authorization Code Flow (Entra ID) and local password login. Issues encrypted, HttpOnly session cookies. Enforces role-based access (<code>admin</code> , <code>shop_admin</code> , <code>user</code>).
Catalog	Renders products grouped by category. Converts prices from the configured base currency to the user's locale currency using stored exchange rates. Supports 25 languages via the <code>i18n</code> engine.
Order	Renders the order form with parameters from global, category, and product parameter sets. Stores orders in PostgreSQL. For DU Admins, triggers the GitLab provisioning webhook immediately; for Project Managers, parks the order for approval.
Approval	Lists pending orders for DU Admins. Approve triggers the GitLab webhook. Reject requires a mandatory comment. Both paths send email notifications and write to the audit log.
Infrastructure	Shows deployed resources grouped by project and environment. Role-scoped: Project Managers see only their own. Triggers the GitLab destroy webhook for decommissioning. Supports “use as template” to pre-fill a new order.

Component	Responsibility
Audit	Append-only log of all system actions. Filterable by date range, user, and action type. Exportable as CSV or PDF.
Notification	Wraps SMTP email delivery. Silently no-ops when SMTP is not configured. SMTP credentials are live-reloadable from the database without a server restart.
GitLab Polling	Goroutine pool that periodically queries the GitLab Pipelines API for in-flight orders. Updates order and infrastructure state on completion or failure, then triggers email notifications. Uses PostgreSQL advisory locks so multiple replicas do not double-poll.
Admin	All Webshop Admin pages: categories, products (CRUD, image upload, AI translation, webhook management, parameter import from <code>variables.tf</code>), environments, GitLab sources, cost centers, currencies, SMTP, AI config, branding, users.
AI Translation	Pluggable interface (<code>aitranslation.Translator</code>). Concrete implementations for Claude, OpenAI, Azure OpenAI, Ollama, and LocalAI. Selected and instantiated at startup from the database configuration; swappable at runtime via the Admin UI.
Repositories	Thin SQL layer using <code>pgx/v5</code> . One repository interface per aggregate root (User, Product, Order, etc.). Implementations in internal/repository/postgres/ .

Table 2.1: Component descriptions

2.4 Level 4 — Key Code Flows

2.4.1 Order Placement Flow (Project Manager)

1. Browser submits `POST /orders/new`.
2. `handler.orderCreate()` validates input, calls `service.OrderService.Create()`.
3. The order service writes the order record with status `pending_approval` to PostgreSQL.
4. The notification service sends an approval-request email to all DU Admins.
5. The audit service writes an `order_placed` entry.
6. Browser redirects to `GET /orders/{id}` showing the *Awaiting Approval* status.

2.4.2 Order Approval & Provisioning Flow

1. DU Admin opens `GET /approvals` and clicks **Approve**.
2. `handler.approvalApprove()` calls `service.OrderService.Approve()`.
3. The order service updates status to **provisioning**, stores the webhook URLs from the product-environment configuration.
4. For each configured webhook (ordered by `exec_order`): an HTTP POST is sent to the GitLab webhook URL with the Linode/Tofu parameters as JSON.
5. GitLab receives the trigger, starts a pipeline, returns a pipeline ID.
6. The webshop stores the pipeline ID(s) in the JSONB `pipeline_ids` column.
7. The GitLab Polling goroutine detects the in-flight pipeline and starts querying the Pipelines API every 30 seconds.
8. When the pipeline completes successfully, the poller sets status to **completed** and sends a deployment-complete email.
9. On failure, status becomes **failed** and failure emails go to the orderer and all DU Admins.

2.4.3 Request–Response Lifecycle (HTMX)

Most user interactions either perform a full page load (navigation) or return an HTML fragment (HTMX). The server detects HTMX requests via the `HX-Request` header. Full renders compose a templ `Layout` component around the page component. Fragments skip the layout and return only the relevant HTML snippet, which HTMX swaps into the DOM.

Chapter 3

Design Decisions & Technology Rationale

3.1 Go as Application Language

Decision: The entire backend is written in Go.

Rationale:

- Compiles to a single self-contained binary with no runtime dependencies — ideal for Docker-based deployment.
- Low memory footprint and predictable garbage collection enable running multiple polling goroutines without resource pressure.
- The standard library `net/http` in Go 1.22+ provides pattern-matching routing (`"GET /path/{id}"`) sufficient for this use case, removing the need for a router dependency.
- `html/template` provides automatic context-aware escaping, eliminating entire classes of XSS vulnerabilities.
- Goroutines provide a natural model for the concurrent GitLab pipeline polling without callback complexity.

Alternative considered: Python/FastAPI with Jinja2. Rejected because the deployment story is more complex (virtualenv, interpreter) and concurrency requires `async`, which increases cognitive overhead.

3.2 Server-Side Rendering with HTMX

Decision: Pages are rendered server-side; dynamic updates use HTMX HTML fragments rather than a JavaScript SPA framework.

Rationale:

- No JSON API surface to design, document, or secure separately — the server owns both data and presentation.
- Progressive enhancement: the UI degrades gracefully if JS is unavailable.
- HTMX's `hx-get/hx-post` attributes on HTML elements cover the interactivity requirements (live status polling, inline form submission, partial page swaps) without React or Vue build pipelines.
- Template-level security: Go's `html/template` auto-escapes all inserted values, preventing XSS by construction.

3.3 templ for Type-Safe HTML Components

Decision: All HTML pages and UI components are authored as `.templ` files and compiled to Go by the `templ` CLI. The `html/template` package is no longer used for page rendering.

Rationale:

- **Compile-time type safety:** Each page receives a concrete typed view struct (e.g. `view.OrderListView`). Passing the wrong data to a component is a compile error, not a runtime `nil` panic or a missing-key silently rendering empty.
- **No `map[string]any`:** The old `html/template` approach required every handler to pass `map[string]any{}` to `h.render()`, providing no compile-time guarantee that the template would find the keys it expected.
- **Composable components:** `templ` supports Go functions as components with typed parameters, children slots, and helper functions — the same mental model as React components but compiled to plain Go without a JS build step.
- **XSS safety preserved:** `templ` auto-escapes all string interpolations by default, maintaining the same security guarantee as `html/template`.
- **IDE support:** The `templ` VS Code extension provides syntax highlighting, autocompletion, and go-to-definition inside `.templ` files.

Design system: A shared component library under `ui/comp/` (`comp.Btn`, `comp.Card`, `comp.Badge`, `comp.Field`, etc.) provides a consistent visual language. Page components under `ui/pages/` compose these primitives and receive typed view models from the handler layer.

3.4 Tailwind CSS without DaisyUI

Decision: Pure Tailwind CSS utility classes; DaisyUI was removed after the migration to `templ` was complete.

Rationale:

- **Theme collision:** DaisyUI ships its own CSS custom properties (`-b1`, `-p`, `-s`, ...) which conflicted with the application's own branding variables (`-bp`, `-bs`). Keeping both required workarounds and caused unpredictable cascade ordering.
- **Specificity fights:** DaisyUI's component classes (`btn`, `badge`, `card`) carry higher specificity than plain utilities, making it hard to override individual properties without `!important` or extra wrapper classes.
- **Bundle size and opacity:** DaisyUI adds several hundred kilobytes of pre-defined component CSS. Tailwind's content-scan only purges *unused* utilities; DaisyUI component styles are included unconditionally, inflating the bundle with styles that the templ components never use.
- **Design independence:** The hand-authored component library in `ui/comp/` uses only Tailwind utilities, making every style decision explicit and auditable. There are no hidden DaisyUI defaults to debug.

What replaced it: The `ui/comp/` package provides the same functional coverage (buttons, badges, cards, form inputs, checkboxes, layout shell) using only Tailwind utilities and the two brand CSS variables `var(-bp)` and `var(-bs)`.

3.5 No Alpine.js

Decision: Alpine.js was removed from the project. The only remaining JavaScript beyond HTMX is the CDN-loaded Alpine.js on the branding admin page, where it powers the live colour preview.

Rationale:

- **All use cases replaced by HTML:** Every place Alpine.js was used — collapsible sections, toggle visibility, show/hide forms — can be handled by the native `<details>/<summary>` element. This is a zero-JavaScript, zero-dependency solution that is accessible by default and degrades gracefully in all browsers.
- **Reduced attack surface:** Alpine.js evaluates expressions from HTML attributes (`x-data`, `x-show`, `x-on:click`). Any XSS that manages to inject an attribute becomes executable code. Removing Alpine.js eliminates this vector.
- **Dependency hygiene:** Alpine.js loaded from a CDN introduces a third-party supply-chain dependency on every page load. Removing it means the only external script is HTMX, which is already trusted as a core dependency.
- **Consistency:** Mixing two reactive JavaScript libraries (HTMX and Alpine) creates a mental model split: developers must decide which tool owns each interactive behaviour. Pure HTMX + HTML is a single coherent model.

Exception: The admin branding page (`/admin/branding`) retains Alpine.js loaded via CDN script tag. It drives a live colour-preview widget where two colour inputs immediately update a mock header element without a server round-trip. This is the one genuine use case for in-page reactive state that cannot be reasonably replaced by `<details>` or HTMX alone.

3.6 PostgreSQL with pgx (No ORM)

Decision: PostgreSQL 15+, accessed via `pgx/v5` with hand-written SQL. No ORM.

Rationale:

- JSONB arrays for `pipeline_ids` allow storing multiple concurrent pipeline references per order without a join table, while retaining full query ability.
- BYTEA columns store product images and the branding logo directly in the database, eliminating a separate object store for this scale.
- Direct SQL gives precise control over query shape and enables `ON CONFLICT DO UPDATE` (upserts) for the branding and app-config singleton rows.
- PostgreSQL advisory locks coordinate the polling goroutines across multiple replicas without an additional message broker.
- `pgx` is the highest-performance PostgreSQL driver for Go; it avoids the `database/sql` reflection overhead.

3.7 Single-Container Architecture

Decision: The entire application (HTTP server, background polling, notification dispatch) runs in a single container.

Rationale: At the anticipated load (tens of users, hundreds of orders per month), microservice decomposition would add operational complexity (service discovery, inter-service auth, distributed tracing) without meaningful benefit. The single-binary model keeps deployment simple: one `docker run` or one Kubernetes `Deployment` object. Horizontal scaling is achieved by running multiple replicas; PostgreSQL advisory locks ensure the polling goroutines do not duplicate work.

3.8 OpenTofu over Terraform

Decision: All IaC modules use OpenTofu 1.9+.

Rationale: OpenTofu is a community-governed, truly open-source fork of Terraform created after HashiCorp changed Terraform's license to BSL 1.1 in 2023. The API, HCL syntax, provider ecosystem, and state format are fully compatible. Using OpenTofu avoids future licensing costs and complies with PORR's open-source software policy.

3.9 GitLab Managed Terraform State

Decision: OpenTofu state is stored in GitLab's built-in HTTP-compatible state backend (GitLab Managed Terraform State).

Rationale:

- Reuses existing GitLab infrastructure — no S3 bucket, no Terraform Cloud account, no Vault setup required.
- State is protected by GitLab’s RBAC: only project members with Developer or higher access can read or modify state.
- State locking via HTTP LOCK/UNLOCK prevents concurrent apply operations.
- State is browsable in the GitLab UI under **Infrastructure** → **Terraform states**.

3.10 Per-Resource Isolated State

Decision: Each VM and each Kubernetes cluster has its own isolated OpenTofu state (named `vm-{label}` and `k8s-{label}`).

Rationale: Shared state across multiple VMs creates blast-radius risk: a failing `apply` for one VM could lock the state and block unrelated VMs. Isolated states mean each resource’s lifecycle is entirely independent. The trade-off is a larger number of state files, which is manageable at the expected scale.

3.11 Webhook-Driven Pipeline Trigger

Decision: The webshop triggers GitLab pipelines via webhook (HTTP POST) rather than the GitLab API’s pipeline-trigger endpoint.

Rationale: Webhooks are the native GitLab mechanism for external triggers. They carry a secure token for authentication and include the ability to pass variables (OpenTofu parameters) as JSON. The pipeline configuration (`.gitlab-ci.yml`) lives in the same repository as the IaC code, making the trigger configuration auditable and version-controlled.

3.12 Internationalization: 25 Languages

Decision: All UI strings are externalized in a key-value `i18n` map covering 25 languages (all EU official languages plus Russian and Ukrainian).

Rationale: PORR AG operates across Europe. The map is defined in `internal/i18n/i18n.go` as a Go map literal for compile-time completeness checking. Translation of product content (not UI strings) is delegated to the optional AI translation service.

3.13 Session Security: Encrypted Cookies

Decision: User sessions are stored in encrypted, signed HTTP cookies — not in the database or server-side session store.

Rationale:

- No database round-trip on every request to look up a session.
- Horizontal scaling is trivial: no shared session store required.
- The `SESSION_KEY` (32-byte, hex-encoded) is used to both sign and encrypt the cookie content via AES-GCM, so the server can verify integrity without a separate HMAC key.
- Cookies are set with `HttpOnly`, `Secure`, and `SameSite=Lax` to mitigate XSS session theft and CSRF.
- Logout invalidates the session by overwriting the cookie with an empty, expired value — no server-side revocation list needed for the MVP threat model.

Alternative considered: Server-side sessions in PostgreSQL. Rejected to avoid the extra table and join on every authenticated request. If revocation of individual sessions at scale is required this can be introduced as a future enhancement.

3.14 Password Hashing: bcrypt

Decision: Local user passwords are hashed with bcrypt (cost factor 12) via golang.org/x/crypto/bcrypt.

Rationale:

- bcrypt is an adaptive, memory-hard function specifically designed for password storage; it is resistant to GPU-accelerated dictionary attacks.
- Cost factor 12 takes 250ms on modern hardware — slow enough to deter brute-force while still acceptable for interactive login.
- The Go standard-library-adjacent `x/crypto` package is actively maintained by the Go team.
- Passwords are never stored in plaintext or with reversible encryption.

Alternative considered: Argon2id. More modern, but bcrypt is widely understood, well-tested in Go, and sufficient for the low login-volume use case.

3.15 Static Asset Embedding: embed.FS

Decision: HTML templates and static assets (CSS, JS, images) are embedded into the Go binary at compile time using the `//go:embed` directive and `embed.FS`.

Rationale:

- The resulting binary is fully self-contained — no external file system paths need to match the deployment environment.
- Deployment is a single `COPY` in the Dockerfile (multi-stage build); no volume mounts are required for templates.

- Template changes are compiled into the binary, so a deployment is always atomic: the binary and its templates are in sync.
- The Go compiler resolves embed paths at build time, making missing-file bugs a compile error rather than a runtime panic.

Trade-off: Hot-reloading templates during development is not possible without restarting the server. The `make dev` target runs the server with `go run`, so a simple restart is fast enough for the development workflow.

Part II

Infrastructure as Code

Chapter 4

GitLab Repository Concept

4.1 Overview

The IaC layer consists of six GitLab repositories in the `porr-ag` group. Each repository is a self-contained OpenTofu module with its own CI/CD pipeline and state. Together they provision all cloud infrastructure on Linode/Akamai.

Table 4.1: GitLab repository overview

Repository	Purpose	Trigger
<code>network-core</code>	VPC, subnets, base firewall, DNS zone	Manual
<code>vm-platform</code>	Linode VM instances (per-VM)	Webhook
<code>k8s-platform</code>	LKE Kubernetes clusters (per-cluster)	Webhook
<code>shared-services</code>	GitLab CE + Vault + Registry	Manual
<code>observability</code>	Grafana + Prometheus + Loki	Manual
<code>infra-state</code>	State backend host (no IaC code)	N/A

4.2 Repository File Structure

Every module repository (except `infra-state`) follows the same file layout:

```
<module>/
+-- .gitlab-ci.yml # CI/CD pipeline definition
+-- backend.tf    # HTTP state backend (GitLab Managed)
+-- provider.tf   # Linode provider + required versions
+-- variables.tf  # Input variable declarations
+-- main.tf       # Resource definitions
+-- outputs.tf    # Output values
```

4.3 State Management

All state files live in the `infra-state` project. The project hosts no IaC code; it is solely a container for GitLab Managed Terraform State files.

State naming convention:

Table 4.2: State name convention per module

Module	State name
network-core	network-core
vm-platform	vm-{vm_label}
k8s-platform	k8s-{cluster_label}
shared-services	shared-prod
observability	observability-prod

The backend URL pattern is:

```
https://<gitlab-host>/api/v4/projects/<INFRA_STATE_PROJECT_ID>/terraform/state/<
↳ STATE_NAME>
```

Backend configuration is passed at `tofu init` time via `-backend-config` flags, not hard-coded in `backend.tf`. This keeps credentials out of version control.

4.4 CI/CD Pipeline Stages

All module pipelines share the same three-stage structure:

1. **validate** — `tofu validate`: checks HCL syntax and provider schema compliance.
2. **plan** — `tofu plan -out=tfplan`: computes the diff and saves the plan artifact.
3. **apply** — `tofu apply tfplan`: applies the plan. For webhook-triggered modules, apply runs automatically on pipeline success. For singleton modules (`shared-services`, `observability`, `network-core`), the apply job is `when: manual` to require deliberate human confirmation.

Destroy flow: Webhook-triggered modules (`vm-platform`, `k8s-platform`) accept a `TF_ACTION=destroy` variable. When set, the apply stage runs `tofu destroy -auto-approve` instead of `tofu apply`.

Listing 4.1 shows the canonical `.gitlab-ci.yml` for the `vm-platform` module. Singleton modules differ only in the apply job: `when: manual` replaces `when: on_success`, and there is no destroy branch.

```
image: ghcr.io/opentofu/opentofu:1.9

variables:
  TF_ACTION: "apply"
```

```
TF_STATE_NAME: "vm-vm-label"
TF_VAR_linode_token: "${LINODE_TOKEN}"

.tofu_init: &tofu_init
  before_script:
    - tofu init
      -reconfigure
      -backend-config="address=${CI_API_V4_URL}/projects/${
        ↳ INFRA_STATE_PROJECT_ID}/terraform/state/${TF_STATE_NAME}"
      -backend-config="lock_address=${CI_API_V4_URL}/projects/${
        ↳ INFRA_STATE_PROJECT_ID}/terraform/state/${TF_STATE_NAME}/lock"
      -backend-config="unlock_address=${CI_API_V4_URL}/projects/${
        ↳ INFRA_STATE_PROJECT_ID}/terraform/state/${TF_STATE_NAME}/lock"
      -backend-config="username=gitlab-ci-token"
      -backend-config="password=${CI_JOB_TOKEN}"
      -backend-config="lock_method=POST"
      -backend-config="unlock_method=DELETE"
      -backend-config="retry_wait_min=5"

  stages:
    - validate
    - plan
    - apply

  validate:
    stage: validate
    <<: *tofu_init
    script:
      - tofu validate
    rules:
      - if: $CI_PIPELINE_SOURCE == "trigger"

  plan:
    stage: plan
    <<: *tofu_init
    script:
      - tofu plan -out=tfplan
    artifacts:
      paths:
        - tfplan
      expire_in: 1 hour
    rules:
      - if: $CI_PIPELINE_SOURCE == "trigger"

  apply:
    stage: apply
    <<: *tofu_init
    script:
      - |
        if [ "${TF_ACTION}" = "destroy" ]; then
          tofu destroy -auto-approve
        else
          tofu apply -auto-approve tfplan
        fi
    dependencies:
      - plan
```

```
rules:
  - if: $CI_PIPELINE_SOURCE == "trigger" && $TF_ACTION == "destroy"
    when: always
  - if: $CI_PIPELINE_SOURCE == "trigger"
    when: on_success
```

Listing 4.1: .gitlab-ci.yml for vm-platform (webhook-triggered module)

Listing 4.2 shows a complete pipeline for singleton modules (`network-core`, `shared-services`, `observability`) where a human must confirm the apply step:

```
image: ghcr.io/opentofu/opentofu:1.9

variables:
  TF_STATE_NAME: "shared-prod"
  TF_VAR_linode_token: "${LINODE_TOKEN}"

.tofu_init: &tofu_init
  before_script:
    - tofu init
      -backend-config="address=${CI_API_V4_URL}/projects/${
        ↪ INFRA_STATE_PROJECT_ID}/terraform/state/${TF_STATE_NAME}"
      -backend-config="lock_address=${CI_API_V4_URL}/projects/${
        ↪ INFRA_STATE_PROJECT_ID}/terraform/state/${TF_STATE_NAME}/lock"
      -backend-config="unlock_address=${CI_API_V4_URL}/projects/${
        ↪ INFRA_STATE_PROJECT_ID}/terraform/state/${TF_STATE_NAME}/lock"
      -backend-config="username=gitlab-ci-token"
      -backend-config="password=${CI_JOB_TOKEN}"
      -backend-config="lock_method=POST"
      -backend-config="unlock_method=DELETE"
      -backend-config="retry_wait_min=5"

  stages:
    - validate
    - plan
    - apply

  validate:
    stage: validate
    <<: *tofu_init
    script:
      - tofu validate
    rules:
      - if: $CI_COMMIT_BRANCH == "main"

  plan:
    stage: plan
    <<: *tofu_init
    script:
      - tofu plan -out=tfplan
    artifacts:
      paths:
        - tfplan
      expire_in: 1 hour
    rules:
      - if: $CI_COMMIT_BRANCH == "main"
```

```
apply:
  stage: apply
  <<: *tofu_init
  script:
    - tofu apply -auto-approve tfplan
  dependencies:
    - plan
  rules:
    - if: $CI_COMMIT_BRANCH == "main"
      when: manual
```

Listing 4.2: .gitlab-ci.yml for singleton modules (manual apply)

Note

The single required CI/CD variable to set in the *infra-state* GitLab project group is `INFRA_STATE_PROJECT_ID` — the numeric project ID of the *infra-state* repository (visible in GitLab under **Settings** → **General**). `CI_JOB_TOKEN` and `CI_API_V4_URL` are provided automatically by GitLab CI. `LINODE_TOKEN` must be set with Read-/Write API access.

4.5 Webhook Integration with the Webshop

1. A DU Admin approves an order in the webshop (or places one directly).
2. For each webhook configured on the product-environment combination, the webshop sends:

```
POST <gitlab-webhook-url>
X-Gitlab-Token: <secret>
Content-Type: application/json

{
  "ref": "main",
  "variables": [
    {"key": "TF_VAR_vm_label", "value": "myvm-01"},
    {"key": "TF_VAR_vpc_id", "value": "12345"},
    ...
  ]
}
```

3. GitLab starts a pipeline for the `main` branch, injecting the variables as environment variables.
4. The pipeline reads `TF_VAR_*` variables, runs `tofu plan` then `tofu apply`.
5. On completion (success or failure), the webshop's polling goroutine detects the status change and updates the order record.

4.6 Dependency Between Modules

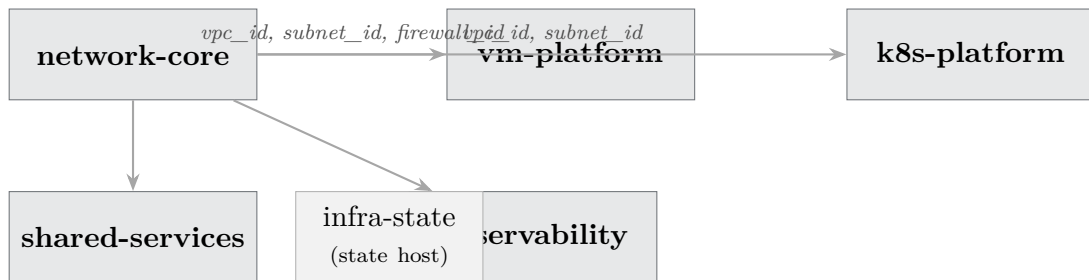


Figure 4.1: Module dependency graph

network-core must be applied first. Its outputs (`vpc_id`, `subnet_ids`, `firewall_id`) are passed as input variables to all other modules.

Chapter 5

OpenTofu Modules

5.1 network-core

Purpose: Foundational network infrastructure. Apply once per environment; no webhook trigger.

Resources created: `linode_vpc` (VPC), `linode_vpc_subnet` (subnets via `for_each`), `linode_firewall` (base inbound/outbound policy), `linode_domain` (optional DNS zone).

Key design choice: Subnets are defined as a `map(object({cidr, description}))` variable. This allows adding subnets by editing a single variable without changing the resource code.

Key outputs: `vpc_id`, `subnet_ids` (map: label $\rightarrow$ ID), `firewall_id`, `dns_zone_id`.

5.2 vm-platform

Purpose: Provision individual Linode VMs on demand. One state per VM (`vm-{label}`). Webhook-triggered by the webshop.

Resources created: `linode_firewall` (VM-specific), `linode_instance`, `linode_firewall_device` (attaches firewall to VM), `linode_volume` (optional block storage), `linode_domain_record` (optional A record).

Key design choice: The VM gets its own firewall (in addition to the base firewall from `network-core`) to allow per-VM port rules without affecting the shared base policy.

Destroy flow: Triggered by the webshop decommission action. `TF_ACTION=destroy` $\rightarrow$ `tofu destroy -auto-approve`.

5.3 k8s-platform

Purpose: Provision Linode Kubernetes Engine (LKE) clusters. One state per cluster (`k8s-{label}`). Webhook-triggered.

Resources created: `linode_firewall` (cluster-specific), `linode_lke_cluster` with dynamic `pool` blocks (supports multiple node pools with optional autoscaler), `linode_domain_record` (optional).

Key output: `kubeconfig` (base64-encoded, marked `sensitive`) — retrieve with `tofu output -raw kubeconfig | base64 -d`.

Node pool design: Node pools are a `list(object)` variable with an optional `autoscaler` sub-object, allowing any combination of fixed-size and auto-scaling pools in a single cluster.

5.4 shared-services

Purpose: Long-lived shared services: GitLab CE, HashiCorp Vault, container registry. Single state (`shared-prod`). Manual apply.

Resources created: Two `linode_instance` resources (GitLab, Vault), two `linode_volume` resources (data volumes), one `linode_firewall` with tailored inbound rules (HTTP/HTTPS/Git SSH for GitLab; Vault API 8200 restricted to RFC1918), two `linode_firewall_device` attachments, optional DNS A records.

Operational note: GitLab CE and Vault are installed via post-provisioning scripts (cloud-init or Ansible), not managed by OpenTofu. OpenTofu only provisions the infrastructure layer.

5.5 observability

Purpose: Grafana, Prometheus, and Loki on a single VM. Single state (`observability-prod`). Manual apply.

Resources created: `linode_instance`, `linode_volume` (200 GB default for metrics and logs), `linode_firewall` (Grafana port configurable, Prometheus/Loki restricted to RFC1918), optional DNS A record.

Key output: `grafana_url` — `http://{ip}:{port}`.

Part III

Administrator Guide

Chapter 6

Initial Setup & Installation

6.1 Prerequisites

- Docker 24+ (or a Kubernetes cluster)
- PostgreSQL 15+ (external or containerized)
- SMTP server credentials (optional, can be configured later)
- Microsoft Entra ID application registration for OIDC (optional for local-only operation)
- Linode API token with Read/Write scope
- GitLab instance with webhook support

6.2 Environment Variables

The webshop is configured entirely via environment variables. Copy `.env.example` to `.env` and populate the following:

Table 6.1: Required environment variables

Variable	Description	Required
<code>DATABASE_URL</code>	PostgreSQL DSN	Yes
<code>SESSION_KEY</code>	32-byte hex session encryption key	Yes
<code>ADMIN_EMAIL</code>	Initial Webshop Admin email	Yes
<code>ADMIN_PASSWORD</code>	Initial Webshop Admin password	Yes
<code>BASE_URL</code>	Public URL of the webshop	Yes
<code>OIDC_CLIENT_ID</code>	Entra ID application client ID	No
<code>OIDC_CLIENT_SECRET</code>	Entra ID client secret	No
<code>OIDC_ISSUER</code>	Entra ID issuer URL	No
<code>SMTP_HOST</code>	SMTP host (overridable in Admin UI)	No
<code>SMTP_PORT</code>	SMTP port (default 587)	No
<code>SMTP_FROM</code>	Sender address	No
<code>SMTP_USERNAME</code>	SMTP username	No
<code>SMTP_PASSWORD</code>	SMTP password	No
<code>AI_PROVIDER</code>	AI translation provider (startup only)	No

Note

SMTP and AI translation can also be configured (and overridden) through the Admin UI at `/admin/smtp` and `/admin/ai-config`. Database settings take precedence over environment variables at runtime.

6.3 Deployment on a Docker Host

This section describes a production-grade deployment on a single Linux host using Docker and an Nginx reverse proxy with TLS termination.

Architecture

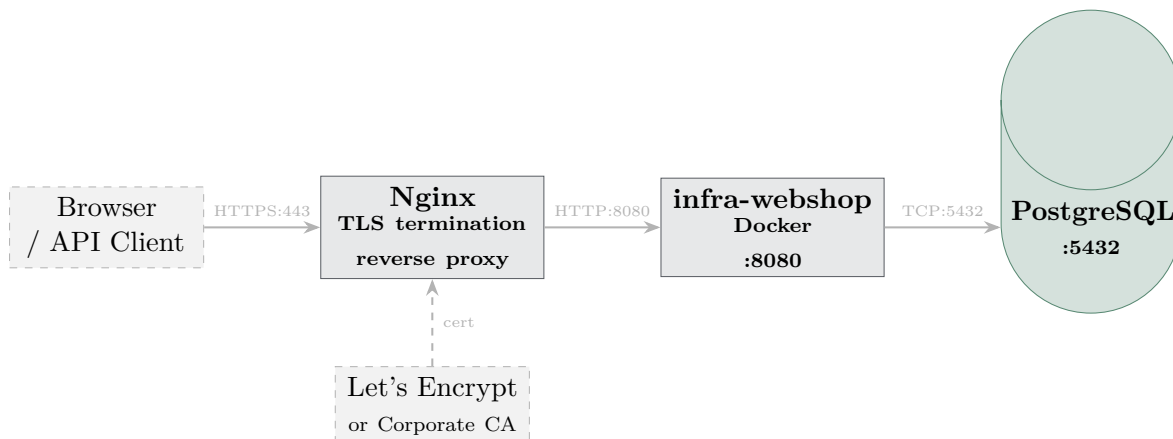


Figure 6.1: Docker Host deployment architecture

Step-by-Step Deployment Procedure

1. Install Docker and Docker Compose.

```
# Debian/Ubuntu
apt-get install -y docker.io docker-compose-v2
systemctl enable --now docker
```

2. Clone the repository and build the image.

```
git clone https://gitlab.example.com/porr-ag/infra-webshop.git
cd infra-webshop
make docker-build
# or pull from registry:
docker login registry.gitlab.example.com
docker pull registry.gitlab.example.com/porr-ag/infra-webshop:latest
```

3. Create the environment file.

```
cp infra/docker-host/.env.example infra/docker-host/.env
# Edit .env with a text editor; set at minimum:
# DATABASE_URL, SESSION_KEY, ADMIN_EMAIL, ADMIN_PASSWORD, BASE_URL
```

4. Start PostgreSQL and the application.

```
cd infra/docker-host
docker compose up -d
```

5. Run database migrations.

```
docker exec infra-webshop /migrate
```

6. Obtain a TLS certificate.

Option A — Let's Encrypt (public domain):

```
apt-get install -y certbot python3-certbot-nginx
certbot --nginx -d shop.example.com
```

Option B — Corporate CA (internal PKI): Place the signed certificate and key at the paths referenced in [infra/docker-host/nginx.conf](#) (see below) and restart Nginx.

7. Configure Nginx as TLS-terminating reverse proxy.

Listing 6.1 shows a minimal but production-ready Nginx configuration:

```
server {
    listen 80;
    server_name shop.example.com;
    return 301 https://$host$request_uri;
}

server {
    listen 443 ssl http2;
```

```
server_name shop.example.com;

ssl_certificate /etc/letsencrypt/live/shop.example.com/fullchain.pem;
ssl_certificate_key /etc/letsencrypt/live/shop.example.com/privkey.pem;
ssl_protocols TLSv1.2 TLSv1.3;
ssl_ciphers HIGH:!aNULL:!MD5;

client_max_body_size 10m;

location / {
    proxy_pass http://127.0.0.1:8080;
    proxy_set_header Host $host;
    proxy_set_header X-Real-IP $remote_addr;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    proxy_set_header X-Forwarded-Proto $scheme;
    proxy_read_timeout 60s;
}
```

Listing 6.1: infra/docker-host/nginx.conf

8. Reload Nginx.

```
nginx -t && systemctl reload nginx
```

9. Verify the deployment.

Navigate to <https://shop.example.com> and log in with **ADMIN_EMAIL** / **ADMIN_PASSWORD**. Change the password immediately under **Settings** → **My Profile**.

10. (Optional) Enable automatic certificate renewal.

```
# Certbot installs a systemd timer automatically
systemctl status certbot.timer
```

Backup & Restore

```
# Backup PostgreSQL
docker exec porr-postgres pg_dump -U webshop webshop \
| gzip > backup-$(date +%Y%m%d).sql.gz

# Restore
zcat backup-20260101.sql.gz \
| docker exec -i porr-postgres psql -U webshop webshop
```

6.4 Deployment on Kubernetes

This section covers deploying the webshop to an existing Kubernetes cluster (e.g. the LKE cluster provisioned by k8s-platform).

Architecture

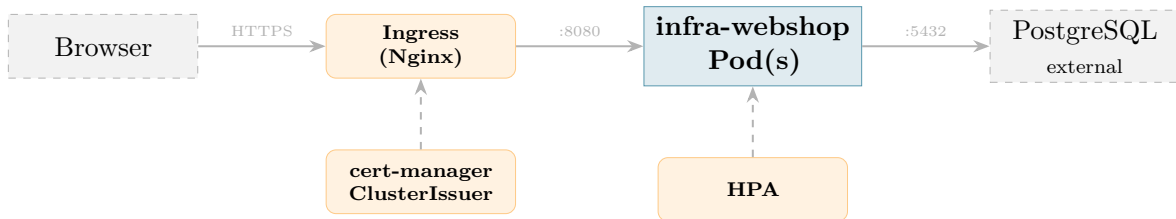


Figure 6.2: Kubernetes deployment architecture

Prerequisites

- `kubectl` and `helm` (v3.12+) configured for the target cluster
- Nginx Ingress Controller installed (`ingress-nginx`)
- `cert-manager` installed with a `ClusterIssuer` for Let's Encrypt or a corporate CA
- The image `porrag/infra-webshop:tag` is publicly available on Docker Hub — no `imagePullSecret` required

Deployment with Helm

The repository ships a Helm chart at [infra/helm/infra-webshop/](#). It covers the full deployment in a single command and handles database migrations automatically as a pre-install/pre-upgrade hook.

```

helm upgrade --install infra-webshop \
  infra/helm/infra-webshop \
  --namespace infra-webshop \
  --create-namespace \
  --set env.databaseUrl="postgres://user:pass@pg.example.com:5432/webshop?sslmode
    ↪ =require" \
  --set env.sessionSecret="<32-byte-hex>" \
  --set env.adminPassword="<initial-password>" \
  --set ingress.hosts[0].host=shop.example.com \
  --set ingress.tls[0].hosts[0]=shop.example.com \
  --set ingress.tls[0].secretName=infra-webshop-tls
  
```

Listing 6.2: Helm install with required values

Note

Sensitive values should be passed via a separate `secrets.values.yaml` file (not committed to version control) rather than as `-set` arguments, so they do not appear in shell history.

```

env:
  databaseUrl: "postgres://user:pass@pg.example.com:5432/webshop?sslmode=require"
  
```

```

sessionSecret: "
  ↪ aabbccddeeff00112233445566778899aabbccddeeff00112233445566778899"
adminPassword: "change-me-immediately"
entra:
  tenantId: "xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx"
  clientId: "xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx"
  clientSecret: "your-client-secret"
  redirectUrl: "https://shop.example.com/auth/callback"

```

Listing 6.3: Example secrets.values.yaml (do not commit)

```

helm upgrade --install infra-webshop infra/helm/infra-webshop \
  --namespace infra-webshop --create-namespace \
  -f values.production.yaml \
  -f secrets.values.yaml

```

Table 6.2 lists the most important configurable values.

Table 6.2: Key Helm values

Key	Description	Default
image.repository	Docker Hub image name	porrag/infra-webshop
image.tag	Image tag (empty = appVersion)	""
replicaCount	Deployment replicas	2
env.databaseUrl	PostgreSQL DSN (required)	—
env.sessionSecret	Session encryption key (req.)	—
env.adminPassword	Initial admin password (req.)	—
ingress.hosts	Ingress host list	shop.example.com
ingress.tls	TLS configuration	(see values.yaml)
autoscaling.enabled	Enable HPA	false
autoscaling.maxReplicas	HPA max replica count	6
migrate.enabled	Run DB migrations as hook	true

Step-by-Step Manual Deployment (without Helm)

1. Create the namespace.

```
kubectl create namespace infra-webshop
```

2. Create the image pull secret (if using a private registry).

```

kubectl create secret docker-registry gitlab-registry \
  --namespace infra-webshop \
  --docker-server=registry.gitlab.example.com \
  --docker-username=<deploy-token-name> \
  --docker-password=<deploy-token-value>

```

3. Create the application Secret.


```

apiVersion: v1
kind: Secret
metadata:
  name: infra-webshop-env
  namespace: infra-webshop
type: Opaque
stringData:
  DATABASE_URL: "postgres://webshop:password@pg.example.com:5432/webshop?
    ↪ sslmode=require"
  SESSION_KEY: "000102030405060708090
    ↪ a0b0c0d0e0f101112131415161718191a1b1c1d1e1f"
  ADMIN_EMAIL: "admin@example.com"
  ADMIN_PASSWORD: "changeme"
  BASE_URL: "https://shop.example.com"

```

Listing 6.4: Kubernetes Secret for the webshop

4. Run database migrations as a Kubernetes Job.

```

apiVersion: batch/v1
kind: Job
metadata:
  name: infra-webshop-migrate
  namespace: infra-webshop
spec:
  template:
    spec:
      restartPolicy: Never
      imagePullSecrets:
        - name: gitlab-registry
      containers:
        - name: migrate
          image: registry.gitlab.example.com/porr-ag/infra-webshop:latest
          command: ["/migrate"]
          envFrom:
            - secretRef:
                name: infra-webshop-env

```

Listing 6.5: Migration Job

```

kubectl apply -f migrate-job.yaml
kubectl wait --for=condition=complete job/infra-webshop-migrate \
  -n infra-webshop --timeout=120s

```

5. Deploy the application.

```

apiVersion: apps/v1
kind: Deployment
metadata:
  name: infra-webshop
  namespace: infra-webshop
spec:
  replicas: 2
  selector:
    matchLabels:

```

```
    app: infra-webshop
  template:
    metadata:
      labels:
        app: infra-webshop
    spec:
      imagePullSecrets:
        - name: gitlab-registry
      containers:
        - name: app
          image: registry.gitlab.example.com/porr-ag/infra-webshop:latest
          ports:
            - containerPort: 8080
          envFrom:
            - secretRef:
                name: infra-webshop-env
          livenessProbe:
            httpGet:
              path: /health
              port: 8080
              initialDelaySeconds: 10
              periodSeconds: 30
          readinessProbe:
            httpGet:
              path: /health
              port: 8080
              initialDelaySeconds: 5
              periodSeconds: 10
          resources:
            requests:
              cpu: "100m"
              memory: "128Mi"
            limits:
              cpu: "500m"
              memory: "512Mi"
      ---
    apiVersion: v1
    kind: Service
    metadata:
      name: infra-webshop
      namespace: infra-webshop
    spec:
      selector:
        app: infra-webshop
      ports:
        - port: 80
          targetPort: 8080
```

Listing 6.6: Deployment and Service

6. Configure the Ingress with TLS.

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: infra-webshop
```

```

namespace: infra-webshop
annotations:
  nginx.ingress.kubernetes.io/proxy-body-size: "10m"
  cert-manager.io/cluster-issuer: "letsencrypt-prod"
spec:
  ingressClassName: nginx
  tls:
    - hosts:
        - shop.example.com
      secretName: infra-webshop-tls
  rules:
    - host: shop.example.com
      http:
        paths:
          - path: /
            pathType: Prefix
            backend:
              service:
                name: infra-webshop
                port:
                  number: 80

```

Listing 6.7: Ingress with cert-manager TLS

7. (Optional) Configure Horizontal Pod Autoscaler.

```

apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: infra-webshop
  namespace: infra-webshop
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: infra-webshop
  minReplicas: 2
  maxReplicas: 6
  metrics:
    - type: Resource
      resource:
        name: cpu
        target:
          type: Utilization
          averageUtilization: 70

```

Listing 6.8: HorizontalPodAutoscaler

8. Apply all manifests and verify.

```

kubectl apply -f secret.yaml
kubectl apply -f deployment.yaml
kubectl apply -f ingress.yaml
kubectl apply -f hpa.yaml # optional

kubectl rollout status deployment/infra-webshop -n infra-webshop

```

```
kubectl get ingress -n infra-webshop
```

Note

The polling goroutines that track GitLab pipeline status run inside each pod. With multiple replicas, advisory locks in PostgreSQL (`pg_try_advisory_lock`) ensure that only one pod polls each pipeline at a time, preventing duplicate status updates.

6.5 Database Migrations

Migrations are versioned SQL files in `internal/migrations/`. Run migrations before starting the server:

```
# Run all pending migrations
make migrate

# Or directly
go run ./cmd/migrate
```

Migrations are idempotent (IF NOT EXISTS, IF NOT EXISTS guards). Re-running is safe.

6.6 First Login

1. Navigate to the webshop URL.
2. Click **Local Login** (not **Sign in with Microsoft**).
3. Enter the email and password from `ADMIN_EMAIL` and `ADMIN_PASSWORD`.
4. Navigate to **Settings** → **My Profile** and change the default password immediately.

Chapter 7

System Configuration

7.1 GitLab Sources

Under **Administration** → **GitLab Sources**:

1. Click **Add New Source**.
2. Enter a **Name** (label), the instance **URL**, and a **Personal Access Token** with `read_api` scope.
3. Save. The source is now available when creating environments and when browsing repositories for parameter import.

7.2 Deployment Environments

Under **Administration** → **Deployment Environments**:

1. Click **Add New Environment**.
2. Enter a **Name** (e.g. *Linode Frankfurt Production*), select the **GitLab Source**, and enter the **Webhook URL** and **Webhook Token**.
3. Save. The environment becomes available when assigning products.

Note

The Webhook URL points to a specific GitLab project pipeline trigger endpoint, e.g.:

<https://gitlab.example.com/api/v4/projects/42/ref/main/trigger/pipeline>

7.3 SMTP Configuration

Under **Administration** → **Email** (</admin/smtp>):

Enter host, port, sender address, username, password, and TLS mode. Click **Save**. The notification service reloads credentials immediately without a server restart.

7.4 AI Translation

Under **Administration** → **AI Translation** (</admin/ai-config>):

1. Select a **Provider**: Claude, OpenAI, Azure OpenAI, Ollama, or LocalAI.
2. Enter the **API endpoint** and **API key**. For Ollama/LocalAI, the endpoint is a local URL and no key is required.
3. Enter the **model name** (e.g. `claude-opus-4-7`, `gpt-4o`, `llama3.1`).
4. Click **Save**. The translator instance is replaced immediately — the next translation request uses the new provider.

7.5 Branding

Under **Administration** → **Shop Design** (</admin/branding>):

- **Shop Name** and **Subtitle** — shown in the header.
- **Primary Color** — header, footer, navigation (default: `#131921`).
- **Accent Color** — buttons and call-to-action elements (default: `#febd69`).
- **Logo** — PNG or SVG, displayed in the header; replaces the shop name text if present.
- **Imprint** — HTML-formatted legal imprint text, shown at </impressum>.

All branding values are stored in a singleton database row and cached in-memory with a mutex for read performance.

7.6 Currencies

Under **Administration** → **Currencies**:

1. The **base currency** (default EUR) is the currency in which all product prices are stored.
2. Optionally configure an exchange rate API key and endpoint.
3. Click **Update Rates** to fetch live rates.
4. Individual rates can be overridden manually for fixed-rate environments.

Prices are displayed in the user's locale currency by looking up the browser locale and applying the stored rate.

Chapter 8

User Management

8.1 Roles

Table 8.1: User roles and capabilities

Role	Description	Auth method	DB role
DU Admin	Full access; direct ordering; approval	SSO	admin
Webshop Admin	Catalog & config; no ordering	Local	shop_admin
Project Manager	Orders own infra; approval required	SSO	user

8.2 Creating a Local Account

Under **Administration** → **Users**:

1. Click **New User**.
2. Enter **Name**, **Email**, **Password**, and select a **Role**.
3. Click **Save**.

SSO users are created automatically on first login.

8.3 Editing Users

Click **Edit** next to a user to change name, email, or role. The password can only be changed by the user themselves under **Settings** → **My Profile**.

8.4 Deactivating Users

Click **Deactivate** on the user edit page. Deactivated users cannot log in. Their orders and projects remain visible in the system.

Chapter 9

Audit Log

The audit log under `/audit` is an append-only, tamper-evident record of all system actions.

9.1 Logged Events

Table 9.1: Audited actions

Action	Trigger
order_placed	User submits an order
order_approved	DU Admin approves
order_rejected	DU Admin rejects (with comment)
order_deployed	Pipeline completes successfully
order_failed	Pipeline fails
decommission_started	User triggers decommission
decommission_complete	Destroy pipeline succeeds
user_login	Successful authentication
user_created	Admin creates a user
config_changed	System configuration updated

9.2 Filtering

Use the filter form to narrow results by:

- **Date range** (from / to)
- **User**
- **Action type**

9.3 Export

Click **Export CSV** or **Export PDF** to download the currently filtered audit log for compliance reporting.

Part IV

Developer Guide

Chapter 10

Development Environment

10.1 Prerequisites

- Go 1.23+
- Node.js 20+ (for Tailwind CSS compilation)
- Docker & Docker Compose (for local PostgreSQL)
- `golangci-lint` (static analysis — installed via `make install-requirements`)
- `pdflatex` or `lualatex` (for handbook compilation, optional)

On Ubuntu or Manjaro all prerequisites except `pdflatex` can be installed with a single command:

```
make install-requirements
```

This detects the native package manager (`pacman` or `apt-get`), installs Go, Node.js, npm, Docker, and Docker Compose, and then installs the Go tools `goimports` and `golangci-lint` via `go install`. A re-login is required afterwards for the Docker group membership to take effect.

Note

Ubuntu note: The `golang-go` package in the Ubuntu repositories may lag behind the latest Go release. If Go 1.23+ is not available, install it manually from <https://go.dev/dl/>.

10.2 Quick Start

```
# Clone and enter the repo
git clone git@gitlab.example.com:porr-ag/infra-webshop.git
cd infra-webshop

# Install all required tools (Ubuntu / Manjaro)
make install-requirements
```

```
# Copy and populate environment
cp .env.example .env
# Edit .env with your local DB credentials

# Start PostgreSQL and the server in one command
make dev

# In another terminal: CSS hot-reload
make css-watch
```

`make dev` starts Docker Compose (PostgreSQL), runs CSS build, then starts the Go server with live environment variables from `.env`.

10.3 Makefile Targets

Table 10.1: Makefile targets

Target	Action
<code>make build</code>	Build CSS + Go binary
<code>make run</code>	Start server (requires running DB)
<code>make migrate</code>	Run database migrations
<code>make css</code>	Build Tailwind CSS once
<code>make css-watch</code>	Watch and rebuild CSS on change
<code>make test</code>	Run all Go tests
<code>make vet</code>	Run <code>go vet</code>
<code>make lint</code>	Run <code>golangci-lint</code>
<code>make install-requirements</code>	Install system packages + Go tools (Ubuntu/Manjaro)
<code>make docker-build</code>	Build Docker image
<code>make dev</code>	Start DB + server (development)
<code>make dev-down</code>	Stop DB + server
<code>make clean</code>	Remove build artefacts
<code>make docs</code>	Compile the LaTeX handbook to PDF

The repository also includes a local smoke test runner in `cmd/smoke/`. Run:

```
DATABASE_URL=postgres://postgres:postgres@localhost:5432/infrashop?sslmode=
  ↪ disable \
go run ./cmd/smoke
```

This command creates temporary rows and verifies cascading cleanup for product deletion.

Chapter 11

Codebase Structure

11.1 Directory Layout

```
infra-webshop/
+--- cmd/
|   +--- server/      # main.go -- server entry point
|   +--- migrate/     # main.go -- migration runner
|   +--- smoke/       # main.go -- smoke test runner
+--- docs/            # Documentation (this handbook)
|   +--- handbook.tex
|   +--- handbook.pdf # compiled output
+--- internal/
|   +--- aitranslation/ # AI translator interface + backends
|   +--- audit/         # AuditService implementation
|   +--- auth/          # OIDC, session store, password hashing
|   +--- config/        # Config struct loaded from env
|   +--- exchange/      # Currency exchange rate service
|   +--- handler/       # HTTP handlers and route registration
|   +--- i18n/          # 25-language key-value translation map
|   +--- migrations/    # SQL migration files (001_*.sql, ...)
|   +--- model/         # Shared domain model structs
|   +--- notification/  # SMTP notification service
|   +--- repository/    # Repository interfaces
|   |   +--- postgres/  # pgx/v5 implementations
|   +--- service/       # Service interfaces
|   |   +--- impl/      # Service implementations
+--- ui/
    +--- comp/          # Shared templ component library
    |   +--- layout.templ # Full-page layout shell + nav
    |   +--- badge.templ  # Status badges
    |   +--- button.templ # Button variants (Btn, BtnLink, IconBtn)
    |   +--- card.templ   # Card, CardBody, CardHeader, PageHeader
    |   +--- form.templ   # Field, Input, Textarea, Select, Checkbox
    +--- pages/          # One package per route group
        +--- admin/       # *.templ for all /admin/* pages
        +--- approvals/
        +--- audit/
        +--- auth/
        +--- catalog/
```

```

|   +-- home/
|   +-- infra/
|   +-- misc/
|   +-- orders/
|   +-- projects/
|   +-- settings/
+-- static/
|   +-- css/style.css      # Compiled Tailwind output
+-- ui.go                  # embed.FS declarations

```

11.2 Layered Architecture

The code follows a strict layered architecture with downward-only dependencies:

Handler → Service → Repository → PostgreSQL

Handler layer ([internal/handler/](#)) HTTP handlers only. Read request, call service, render template. No business logic. No SQL.

Service layer ([internal/service/](#)) Business rules and orchestration. Calls repositories. Triggers notifications, audit writes, webhook dispatches. Defined as interfaces in [service.go](#), implemented in [impl/](#).

Repository layer ([internal/repository/](#)) Data access. Defined as interfaces (one per aggregate root). Implemented in [postgres/](#) using pgx. No business logic.

Model layer ([internal/model/](#)) Plain Go structs shared across layers. No methods beyond simple helpers.

11.3 Template System (templ)

All pages are authored as `.templ` files and compiled to Go by `templ generate ./....`. Each `.templ` file produces a corresponding `_templ.go` file which is committed alongside the source.

Page components receive a typed view struct:

```

// ui/pages/orders/orders.templ
templ OrderList(vd view.OrderListView) {
    @comp.Layout(vd.PageData) {
        // page content -- type-checked at compile time
    }
}

```

Handlers assemble the view struct and call `renderTempl`:

```

func (h *Handler) orderList(w http.ResponseWriter, r *http.Request) {
    orders, _ := h.orders.ListByUser(r.Context(), sess.UserID)
    renderTempl(w, r, orderpages.OrderList(view.OrderListView{
        PageData: h.buildPageData(w, r),
    }

```

```
    Orders: orders,
  )))
}
```

`buildPageData` assembles the common `view.PageData` struct providing:

- .Lang** User's active language code (e.g. "de").
- .Brand** Branding values (colours, shop name, logo flag).
- .Session** Current user session (name, role, email).
- .Flash** One-time flash message (success/error).
- .SearchQuery** Current catalog search term from the URL.

Translations are called directly in templ files:

```
{ i18n.T("orders.title", vd.Lang) } // translated string
{ string(o.Status) }                // named type requires cast
```

11.4 Design System

All UI is built from a small, self-contained component library under `src/ui/comp/`. Components are plain templ functions — no framework, no runtime dependencies.

Design Tokens

Two CSS custom properties drive every coloured surface:

var(--bp) Brand primary colour (default `#131921`, the dark header). Used for header, navigation, footer, active highlights, and price text.

var(--bs) Brand secondary / accent colour (default `#febd69`, amber). Used for CTA buttons, active language badges, and focus rings.

The values are injected by `comp.Layout` as inline CSS custom properties on `<html>`. Every component should reference these variables rather than hardcoding colours:

```
<!-- correct -->
<div style="background-color:var(--bp)">...</div>
<a style="color:var(--bs)">...</a>

<!-- wrong: breaks custom branding -->
<div class="bg-[#131921]">...</div>
```

Layout Shell (`comp.Layout`)

Every page is wrapped in `comp.Layout`:

```
templ MyPage(vd view.MyView) {
  @comp.Layout(vd.PageData) {
    // page content here
  }
}
```



```
    }
  }
```

`comp.Layout` renders: `<head>` with meta tags and stylesheets, sticky header with brand logo / search bar / user menu / language picker, the top navigation bar with role-based links, a flash banner slot, the `<main>` content area (2XL max-width, horizontal padding), and the footer. Nothing else should set `<html>`, `<head>`, or `<body>`.

Page Structure

`comp.PageHeader(title, backHref)` Full-width page title row with an optional back arrow link. Use at the top of every dedicated page.

`comp.PageHeaderAction(title, backHref, action)` Same as above but with an action slot on the right (e.g. a primary button).

`comp.Section(title)` A titled content group with a subtle separator. Use to divide a page into logical parts.

`comp.EmptyState(message)` Centred placeholder message for empty lists or zero results.

```
@comp.PageHeaderAction("Bestellungen", "/", action) {
  @comp.BtnLink("Neue Bestellung", "/catalog",
    comp.VPrimary, comp.SzMD)
}
@comp.Section("Offene Bestellungen") {
  if len(orders) == 0 {
    @comp.EmptyState("Keine Bestellungen vorhanden.")
  }
  // list rows...
}
```

Cards

`comp.Card()` White surface with border and subtle shadow. Use as a generic content container.

`comp.CardBody()` Adds consistent padding inside a `Card`.

`comp.CardHeader(title)` A titled header strip for a card.

```
@comp.Card() {
  @comp.CardHeader("Details") {}
  @comp.CardBody() {
    // form fields, table, prose, etc.
  }
}
```

Buttons

comp.Btn(label, variant, size, templ.Attributes) A `<button>` element. Pass extra HTML attributes (e.g. `hx-post`, `disabled`) via the fourth argument.

comp.BtnLink(label, href, variant, size) An `<a>` element styled as a button. Use for navigation CTAs.

comp.IconBtn(variant, size, templ.Attributes) Square icon button; children supply the icon SVG.

comp.BtnClass(variant, size) Returns just the Tailwind class string. Use when you need to apply button styling to an element not covered by the components above (e.g. an HTMX-powered `<button>` inside a `templ.children...` slot).

Variants:

<code>comp.VPrimary</code>	Filled, uses <code>var(-bp)</code> background — main CTAs.
<code>comp.VSecondary</code>	Outlined border button — secondary actions.
<code>comp.VGhost</code>	Transparent, text-only — low-emphasis actions.
<code>comp.VDanger</code>	Red filled — destructive actions (delete, reject).

Sizes:

<code>comp.SzXS</code>	Extra small — inline or table-row actions.
<code>comp.SzSM</code>	Small — secondary actions, compact layouts.
<code>comp.SzMD</code>	Medium (default) — primary page-level CTAs.

```
// Primary CTA
@comp.BtnLink("Jetzt bestellen", "/catalog/"+id,
              comp.VPrimary, comp.SzMD)

// Danger action with HTMX
@comp.Btn("Ablehnen", comp.VDanger, comp.SzSM,
          templ.Attributes{"hx-delete": "/orders/"+id,
                           "hx-confirm": "Wirklich ablehnen?"})

// Icon button (custom SVG inside)
@comp.IconBtn(comp.VGhost, comp.SzXS,
              templ.Attributes{"aria-label": "Bearbeiten"}) {
  <svg .../>
}
```

Badges

comp.Badge(label, color) A small inline pill. Use `comp.BadgeGreen`, `comp.BadgeRed`, `comp.BadgeYellow`, `comp.BadgeBlue`, or `comp.BadgeGray` for the color argument.

comp.StatusBadge(status) Convenience wrapper that maps an order/resource status string to the appropriate colour via `comp.StatusBadgeColor`. No need to branch on status in the template.

```
// Generic badge
@comp.Badge("Aktiv", comp.BadgeGreen)
```

```
@comp.Badge("Fehler", comp.BadgeRed)

// Status-aware badge (recommended for orders)
@comp.StatusBadge(string(order.Status))
```

Status colour mapping (via `comp.StatusBadgeColor`):

active, approved, completed	Green
pending	Yellow
failed, rejected	Red
provisioning, decommissioning	Blue
Everything else	Gray

Forms

All form fields use the components in `ui/comp/form.templ` for consistent label alignment, spacing, and required-marker rendering.

`comp.Field(label, name, required)` A `<label>` + slot wrapper. Place an `Input`, `Select`, or `Textarea` inside it.

`comp.Input(name, type, placeholder, value, attrs)` Styled `<input>`. Pass extra attributes (e.g. `hx-get`, `min`, `step`) via the last argument.

`comp.Textarea(name, placeholder, value, rows, attrs)` Styled `<textarea>`.

`comp.Select(name, attrs)` Styled `<select>` wrapper; children are `<option>` elements written inline.

`comp.Checkbox(name, label, checked)` A labelled checkbox with correct `for`/`id` wiring.

```
<form method="POST" action="/admin/products">
  @comp.Field("Produktname", "name", true) {
    @comp.Input("name", "text", "z.B. VM-Platform",
      product.Name, nil)
  }
  @comp.Field("Beschreibung", "description", false) {
    @comp.Textarea("description", "", product.Description,
      4, nil)
  }
  @comp.Field("Kategorie", "category_id", true) {
    @comp.Select("category_id", nil) {
      for _, c := range categories {
        <option value={ fmt.Sprint(c.ID) }
          selected?={ c.ID == product.CategoryID }>
          { c.Name }
        </option>
      }
    }
  }
  @comp.Field("Aktiv", "active", false) {
    @comp.Checkbox("active", "Produkt ist sichtbar",
      product.Active)
```

```
    }  
    @comp.Btn("Speichern", comp.VPrimary, comp.SzMD, nil)  
</form>
```

Flash Messages

Flash messages are set server-side via `redirectWithFlash` and rendered automatically by `comp.Layout` from `pd.Flash`. HTMX responses that want to show a flash message use the `HX-Trigger` response header instead:

```
// In a handler that returns an HTMX partial:  
w.Header().Set("HX-Trigger", `{"showFlash":{"kind":"success",  
    "message":"Gespeichert"}}`)
```

Component Conventions

- **Never hardcode colours.** Always use `var(-bp)`, `var(-bs)`, or Tailwind's neutral palette (`slate-*`). Hardcoded hex values break custom branding.
- **Use `comp.VPrimary` for the single most important action** on a page. Use `comp.VSecondary` or `comp.VGhost` for everything else.
- **No JavaScript.** Interactive disclosure (menus, modals, accordions) uses native `<details>/<summary>`. Server-side partial updates use HTMX attributes. Never add `<script>` tags to component files.
- **No inline style except for brand tokens.** Arbitrary colours, sizes, and spacing belong in Tailwind utility classes, not `style=""` attributes.
- **Prefer `comp.StatusBadge` over manual branching.** The mapping is maintained centrally; adding a new status only requires updating `comp.StatusBadgeColor`.

11.5 Adding Translations

All UI strings live in `internal/i18n/i18n.go` as a `map[string]map[string]string`. To add a new key:

```
// In translations map:  
"my.new.key": {  
    "en": "English text",  
    "de": "Deutscher Text",  
    "fr": "Texte francais",  
    // ... all 25 language codes  
},
```

Use in templates with `{{t "my.new.key" .Lang}}`.

11.6 Database Migrations

Migration files follow the naming pattern `NNN_description.sql` and are executed in alphabetical order. Write migrations defensively:

```
ALTER TABLE products
  ADD COLUMN IF NOT EXISTS image_mime TEXT NOT NULL DEFAULT 'image/jpeg';

CREATE TABLE IF NOT EXISTS app_config (
  id INT PRIMARY KEY DEFAULT 1 CHECK (id = 1),
  smtp_host TEXT NOT NULL DEFAULT ''
  -- ...
);
```

Important

Never modify a migration that has already been applied to production. Always create a new migration file for schema changes.

Chapter 12

Adding a New Feature

This chapter describes the canonical pattern for adding a feature end-to-end.

12.1 Example: Adding a New Admin Page

Goal: Add an admin page at `/admin/widgets` that lists and creates widgets stored in a new `widgets` database table.

12.1.1 Step 1: Create the Migration

Create `internal/migrations/010_widgets.sql`:

```
CREATE TABLE IF NOT EXISTS widgets (  
  id BIGSERIAL PRIMARY KEY,  
  name TEXT NOT NULL,  
  active BOOLEAN NOT NULL DEFAULT TRUE,  
  created_at TIMESTAMPTZ NOT NULL DEFAULT now()  
);
```

12.1.2 Step 2: Add the Model

In `internal/model/model.go`:

```
type Widget struct {  
  ID      int64  
  Name    string  
  Active  bool  
  CreatedAt time.Time  
}
```

12.1.3 Step 3: Define the Repository Interface

In `internal/repository/repository.go`:

```
type WidgetRepository interface {
    FindAll(ctx context.Context) ([]model.Widget, error)
    Create(ctx context.Context, name string) error
    Delete(ctx context.Context, id int64) error
}
```

12.1.4 Step 4: Implement the Repository

Create `internal/repository/postgres/widget.go`:

```
type widgetRepo struct{ pool *pgxpool.Pool }

func NewWidgetRepository(pool *pgxpool.Pool) repository.WidgetRepository {
    return &widgetRepo{pool: pool}
}

func (r *widgetRepo) FindAll(ctx context.Context) ([]model.Widget, error) {
    rows, err := r.pool.Query(ctx,
        'SELECT id, name, active, created_at FROM widgets ORDER BY name')
    // ... scan rows into []model.Widget
}
```

12.1.5 Step 5: Add the Handler

Add to the Handler struct in `routes.go`:

```
widgets repository.WidgetRepository
```

Add to Deps struct and wire in `New()`. Add a view struct in `internal/view/pages.go`:

```
type AdminWidgetsView struct {
    view.PageData
    Widgets []model.Widget
}
```

Add handler methods to a new file `handler/widgets.go`:

```
func (h *Handler) adminWidgets(w http.ResponseWriter, r *http.Request) {
    items, _ := h.widgets.FindAll(r.Context())
    renderTempl(w, r, adminpages.AdminWidgets(view.AdminWidgetsView{
        PageData: h.buildPageData(w, r),
        Widgets: items,
    }))
}
```

Register routes in `RegisterRoutes`:

```
mux.Handle("GET /admin/widgets", admin(h.adminWidgets))
mux.Handle("POST /admin/widgets", admin(h.adminWidgetCreate))
```

12.1.6 Step 6: Create the templ Component

Create `ui/pages/admin/widgets.templ`:

```
package admin

import (
    "github.com/porr-ag/infra-webshop/src/internal/i18n"
    "github.com/porr-ag/infra-webshop/src/internal/view"
    "github.com/porr-ag/infra-webshop/src/ui/comp"
)

templ AdminWidgets(vd view.AdminWidgetsView) {
    @comp.Layout(vd.PageData) {
        @comp.PageHeader(i18n.T("admin.widgets", vd.Lang), "/admin")
        @comp.Card() {
            <table class="w-full text-sm">
                <tbody>
                    for _, w := range vd.Widgets {
                        <tr class="hover:bg-slate-50">
                            <td class="px-4 py-3">{ w.Name }</td>
                        </tr>
                    }
                </tbody>
            </table>
        }
    }
}
```

Run `templ generate ./...` to compile to Go, then `go build ./...` to verify type correctness before running.

12.1.7 Step 7: Add i18n Keys

In `internal/i18n/i18n.go`, add the key with all 25 language translations.

12.1.8 Step 8: Wire Up in main.go

```
widgetRepo := pgRepo.NewWidgetRepository(pool)
// ...
h := handler.New(handler.Deps{
    // ...
    Widgets: widgetRepo,
})
```

12.1.9 Step 9: Test

Add test stubs in `internal/repository/` (or use the real DB in integration tests).
Run:

```
make test
make vet
make lint
```


Part V

Project Manager & Product Lifecycle Guide

Chapter 13

Ordering Infrastructure

13.1 The Order Lifecycle

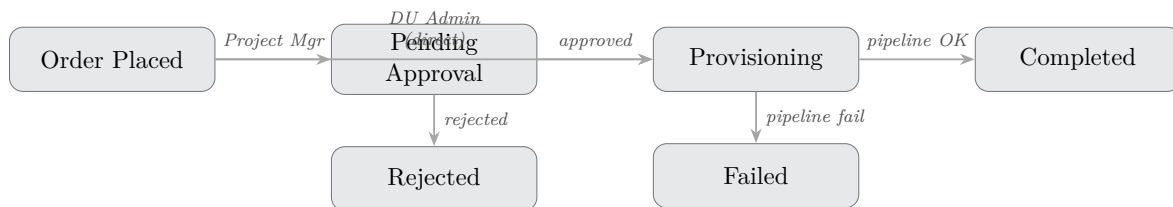


Figure 13.1: Order lifecycle state machine

13.2 Placing an Order

1. Navigate to **Catalog** and select a product.
2. Click **Configure** → select the deployment environment.
3. Select or create a **Project**.
4. Fill in the **Parameters** (required fields are marked).
5. Select or confirm the **Cost Center**.
6. Click **Deploy Now**.

As a **Project Manager** the order status changes to *Pending Approval*. A DU Admin receives an email notification.

As a **DU Admin** the GitLab webhook fires immediately.

13.3 Tracking Status

After placing an order, navigate to **My Orders** → {order}. The status updates live via HTMX polling every 30 seconds.

13.4 Using an Existing Deployment as a Template

1. Open **Infrastructure**.
2. Find the resource and click **Use as Template**.
3. The order form opens pre-filled with the original parameters.
4. Modify as needed and submit.

13.5 Decommissioning

1. Open **Infrastructure**.
2. Find the resource and click **Decommission**.
3. Confirm the action.
4. The destroy webhook fires; status changes to *Decommissioning*.
5. On pipeline completion, status becomes *Decommissioned*.

Cascade Decommissioning on Deletion

Deleting a **project**, **product**, or **category** automatically triggers decommissioning for all active infrastructure elements associated with that object before the database record is removed:

- **Project deleted** — all active infrastructure elements of the project receive a destroy webhook call.
- **Product deleted** — all active infrastructure elements provisioned from that product receive a destroy webhook call.
- **Category deleted** — all active infrastructure elements of every product in the category receive a destroy webhook call.

Infrastructure already in status *Decommissioning* or *Decommissioned* is skipped. Dependent database rows (translations, parameters, environment assignments) are removed automatically via PostgreSQL ON DELETE CASCADE constraints after the webhooks have been triggered.

Important

Deleting a project, product, or category is irreversible. All associated cloud resources will be destroyed by OpenTofu. Verify there are no active workloads before proceeding.

Chapter 14

Introducing a New Product

This chapter describes the complete process for making a new infrastructure product available in the webshop. Follow the steps sequentially.

14.1 Overview

1. Plan the product
2. Prepare the OpenTofu module
3. Configure the GitLab webhook
4. Create the product in the Admin UI
5. Define parameters (import or manual)
6. Configure deployment environments and pricing
7. Set up AI translations
8. Test the full order flow
9. Publish

14.2 Step 1: Plan the Product

Before touching any UI, answer these questions:

- **What does the product deploy?** (e.g. *Managed PostgreSQL VM on Linode*)
- **Which OpenTofu module delivers it?** (vm-platform, k8s-platform, or a new module)
- **In which environments is it available?** (e.g. Linode Frankfurt, Linode Amsterdam)
- **What parameters does the user need to provide?** (e.g. `vm_label`, `instance_type`, `db_name`)

- **What is the base price in EUR per environment?**
- **Who assigns the cost center?** (project / select / overhead)

14.3 Step 2: Prepare the OpenTofu Module

If the product uses an existing module (vm-platform or k8s-platform), skip to Step 3. For a new product type:

1. Create a new GitLab repository in the `porr-ag` group.
2. Add the standard files: `provider.tf`, `backend.tf`, `variables.tf`, `main.tf`, `outputs.tf`, `.gitlab-ci.yml`.
3. Declare all user-provided parameters as `variable` blocks in `variables.tf` with `description`, `type`, and `default` where applicable:

```
variable "db_name" {
  description = "Name of the PostgreSQL database to create."
  type        = string
}

variable "db_size_gb" {
  description = "Database volume size in GB."
  type        = number
  default     = 20
}
```

4. Test the module locally:

```
tofu init -backend-config="address=..." \
          -backend-config="username=..." \
          -backend-config="password=..."
tofu plan -var="linode_token=$TOKEN" \
          -var="db_name=test" \
          -var="db_size_gb=20"
tofu apply
tofu destroy
```

5. Commit and push to the `main` branch.

14.4 Step 3: Configure the GitLab Webhook

In the GitLab repository:

1. Go to **Settings** → **CI/CD** → **Pipeline trigger tokens**.
2. Click **Add new token** and give it a descriptive name (e.g. *webshop-prod*).
3. Note the **trigger URL** and **token**.

The trigger URL has the form:

```
https://gitlab.example.com/api/v4/projects/<PROJECT_ID>/ref/main/trigger/pipeline
```

14.5 Step 4: Create a Category (if needed)

Under **Administration** → **Categories**:

1. Click **New Category**.
2. Enter a **Name**.
3. Optionally assign a **Category Parameter Set** (see Step 5).
4. Save.

14.6 Step 5: Create the Product

Under **Administration** → **Products** → **New**:

14.6.1 Basic Information

1. Select the **Category**.
2. Enter the product **Name** in the primary language.
3. Enter a detailed **Description** in the primary language.
4. Upload a product **Image** (PNG or JPEG, max 10 MB).
5. Click **Save & Continue**.

14.6.2 Parameters

Choose one of two import methods:

Option A — Import from `variables.tf` (recommended):

1. Click **Browse Repository**.
2. Select the **GitLab Source** and the repository.
3. Select the branch (`main`) and navigate to `variables.tf`.
4. Click **Import Parameters**.
5. The webshop parses the HCL and creates parameter records for each variable. Review and adjust:
 - **Required** — must the user fill this in?
 - **Sensitive** — should the value be masked in the UI and audit log?
 - **Default value** — pre-fill the order form.

- **Type** — text, number, boolean, or dropdown.

Option B — Manual entry:

Click **Add Parameter** and fill in each field manually.

Note

Parameters whose names match OpenTofu variable names (e.g. `instance_type`) are passed to the webhook as `TF_VAR_instance_type`. The name in the webshop must match the `variable` block name in `variables.tf` exactly.

14.6.3 Deployment Environments

1. Click **Add Environment**.
2. Select an environment from the dropdown (must be pre-configured under Administration → Environments).
3. Enter the **Price** in the base currency.
4. For the webhook, either:
 - Use the environment's default webhook URL, or
 - Enter a product-specific webhook URL and token.
5. Configure the **Cost Center Mode**:

Project Cost center inherited from the project.

Select Orderer selects from the cost center list.

Overhead Always assigns a fixed cost center.
6. Repeat for each environment.

14.6.4 Multiple Webhooks (Pipeline Arrays)

A single product-environment combination can have multiple webhooks executed in order (e.g. first provision the VM, then configure DNS, then register in Vault). Under the webhook section:

1. Click **Add Webhook**.
2. Enter URL, token, and **Execution Order** (1, 2, 3 ...).
3. Webhooks are triggered sequentially in ascending order.

14.7 Step 6: Configure AI Translation

1. Ensure an AI provider is configured (Chapter 7).
2. Open the product in **Administration** → **Products**.

3. Click **Generate AI Translation**.
4. Review the generated translations for all 25 languages.
5. Edit any that require adjustment.
6. Save.

Tip

Generate translations early in the process. The AI translates both the product name and description. Individual translations can be re-generated for specific languages by editing the translation tab directly.

14.8 Step 7: Global and Category Parameter Sets

Global parameters (under **Administration** → **Global Parameters**) apply to every order regardless of product. Use them for organization-wide fields such as a cost center code or project tag that must always be captured.

Category parameters (under **Administration** → **Categories** → **{category}** → **Parameters**) apply to all products within a category. Use them to avoid repeating parameters shared by related products (e.g. all VM products share a **region** parameter).

Parameter hierarchy (applied in this order on the order form):

1. Global parameters
2. Category parameters
3. Product parameters
4. Environment-specific parameters (override per-environment)

14.9 Step 8: Test the Full Order Flow

1. Log in as a **DU Admin** (DU Admins trigger webhooks directly).
2. Navigate to **Catalog** and find the new product.
3. Click **Configure**, fill in test parameters, and click **Deploy Now**.
4. Monitor the order under **My Orders** — the status should progress from *Provisioning* to *Completed*.
5. Verify the infrastructure was created in the Linode Cloud Manager.
6. Test the email notification (check the admin email inbox).
7. Test the **Decommission** flow:
 - Open **Infrastructure**.
 - Click **Decommission** on the test resource.

- Verify the destroy pipeline runs and the resource disappears from the Linode Cloud Manager.

Important

Always test decommission in a non-production environment first. A destroy pipeline permanently deletes cloud resources and their data.

14.10 Step 9: Test with a Project Manager Account

1. Log in (or create a test account) as a **Project Manager**.
2. Place an order for the new product.
3. Verify the order shows *Pending Approval*.
4. Switch to the **DU Admin** account.
5. Approve the order under **Approvals**.
6. Verify the webhook fires and provisioning completes.
7. Verify the Project Manager receives the confirmation email.

14.11 Step 10: Product is Live

Once testing passes:

- The product is immediately visible in the catalog.
- No deployment or restart of the webshop is required.
- Announce the new product to users via the notification service or the shop's subtitle/branding.

Chapter 15

Project Management

15.1 Projects

A *project* is a container for related infrastructure orders. Every order must be associated with a project. Projects have:

- A **Name** and optional **Description**.
- An optional **Cost Center** (used when cost center mode is *Project*).
- An owner (the user who created it).

Project Managers see only their own projects. **DU Admins** and **Webshop Admins** see all projects.

15.2 Infrastructure Overview

The **Infrastructure** page shows all deployed resources grouped by project and environment. Each resource shows:

- Product name and environment.
- Deployment date and current status.
- Key parameters (e.g. VM label, cluster name).
- Buttons: **Use as Template**, **Decommission**.

15.3 Cost Center Assignment

Cost centers are maintained under **Administration** → **Cost Centers**. The three assignment modes give fine-grained control over who can allocate costs:

Project No user input; cost center comes from the project. Use for products where cost is always allocated to the project budget.

Select Orderer picks from the cost center list. Use when individual orders can span cost centers.

Overhead Always assigns a fixed cost center regardless of order. Use for shared services (monitoring, DNS).

Appendix A

Supported Languages

The following 25 languages are supported in the UI and for AI product translation:

Code	Language	Native name
de	German	Deutsch
en	English	English
fr	French	Français
it	Italian	Italiano
es	Spanish	Español
pt	Portuguese	Português
nl	Dutch	Nederlands
pl	Polish	Polski
cs	Czech	Čeština
sk	Slovak	Slovenčina
hu	Hungarian	Magyar
ro	Romanian	Română
bg	Bulgarian	Balgarski (Cyrillic)
hr	Croatian	Hrvatski
sl	Slovenian	Slovenščina
et	Estonian	Eesti
lv	Latvian	Latviešu
lt	Lithuanian	Lietuvių
fi	Finnish	Suomi
sv	Swedish	Svenska
da	Danish	Dansk
el	Greek	Ellinika (Greek script)
mt	Maltese	Malti
ru	Russian	Russkij (Cyrillic)
uk	Ukrainian	Ukrayinska (Cyrillic)

Table A.1: Supported language codes

Appendix B

Environment Variable Reference

Variable	Description	Required
DATABASE_URL	PostgreSQL connection string (DSN)	Yes
SESSION_KEY	32-byte hex key for session encryption	Yes
ADMIN_EMAIL	Email of the initial Webshop Admin account	Yes
ADMIN_PASSWORD	Password of the initial admin account	Yes
BASE_URL	Public base URL (e.g. <code>https://shop.example.com</code>)	Yes
PORT	HTTP listen port (default: 8080)	No
OIDC_CLIENT_ID	Entra ID application (client) ID	No
OIDC_CLIENT_SECRET	Entra ID client secret	No
OIDC_ISSUER	OIDC issuer URL (<code>https://login.microsoft...</code>)	No
SMTP_HOST	SMTP server hostname	No
SMTP_PORT	SMTP port (default: 587)	No
SMTP_FROM	Sender email address	No
SMTP_USERNAME	SMTP authentication username	No
SMTP_PASSWORD	SMTP authentication password	No
SMTP_TLS	<code>true</code> to use TLS/SSL	No
AI_PROVIDER	AI translation provider at startup	No
AI_ENDPOINT	AI API endpoint URL	No
AI_API_KEY	AI API key	No
AI_MODEL	AI model name	No
EXCHANGE_RATE_KEY	Exchange rate API key	No

Table B.1: Environment variable reference

Appendix C

OpenTofu Module Variable Reference

C.1 network-core Variables

Variable	Type	Description	Default
linode_token	string	Linode API token (sensitive)	—
region	string	Linode region	eu-central
vpc_label	string	VPC label	porr-vpc-prod
vpc_description	string	VPC description	(see defaults)
subnets	map(obj)	Subnet map (label→cidr)	app + mgmt
firewall_inbound_rules	list(obj)	Inbound firewall rules	SSH + ICMP
dns_domain	string	DNS zone domain (empty=none)	""
tags	list	Resource tags	["infra","network-core"]

Table C.1: network-core variables

C.2 vm-platform Variables

Variable	Type	Description	Default
linode_token	string	Linode API token (sensitive)	—
vm_label	string	Unique VM label (state name)	—
region	string	Linode region	eu-central
vpc_id	number	VPC ID from network-core	—
subnet_id	number	Subnet ID from network-core	—
firewall_id	number	Base firewall ID	—
instance_type	string	Linode plan	g6-standard-2
image	string	Boot image	linode/debian12
root_pass	string	Root password (sensitive)	—
ssh_keys	list	Authorized SSH keys	[]
volume_size_gb	number	Block storage size (0=none)	0
additional_inbound_rules	list(obj)	Extra firewall rules	[]
dns_zone_id	number	DNS zone ID (null=none)	null
dns_hostname	string	A record hostname	""

Variable	Type	Description	Default
tags	list	Resource tags	["infra","vm"]

Table C.2: vm-platform variables

C.3 k8s-platform Variables

Variable	Type	Description	Default
linode_token	string	Linode API token (sensitive)	—
cluster_label	string	Unique cluster label	—
region	string	Linode region	eu-central
kubernetes_version	string	K8s version	1.31
vpc_id	number	VPC ID	—
subnet_id	number	Subnet ID	—
node_pools	list(obj)	Node pool configurations	1× g6-standard-2
additional_inbound_rules	list(obj)	Extra firewall rules	[]
dns_zone_id	number	DNS zone ID (null=none)	null
dns_hostname	string	A record hostname	"
tags	list	Resource tags	["infra","k8s"]

Table C.3: k8s-platform variables

C.4 shared-services Variables

Variable	Type	Description	Default
linode_token	string	Linode API token (sensitive)	—
region	string	Linode region	eu-central
gitlab_label	string	GitLab CE Linode label	porr-gitlab
gitlab_instance_type	string	Linode plan for GitLab CE	g6-standard-4
vault_label	string	Vault Linode label	porr-vault
vault_instance_type	string	Linode plan for Vault	g6-standard-2
image	string	Boot image	linode/debian12
root_pass	string	Root password (sensitive)	—
ssh_keys	list	Authorized SSH keys	[]
vpc_id	number	VPC ID from network-core	—
subnet_id	number	Subnet ID from network-core	—
allowed_admin_ipv4	list	IPv4 CIDRs for SSH & Vault admin	["0.0.0.0/0"]
tags	list	Resource tags	["infra","shared"]

Table C.4: shared-services variables

C.5 observability Variables

Variable	Type	Description	Default
linode_token	string	Linode API token (sensitive)	—
region	string	Linode region	eu-central
obs_label	string	Linode instance label	porr-obs
instance_type	string	Linode plan	g6-standard-2
image	string	Boot image	linode/debian12
root_pass	string	Root password (sensitive)	—
ssh_keys	list	Authorized SSH keys	[]
vpc_id	number	VPC ID from network-core	—
subnet_id	number	Subnet ID from network-core	—
volume_size_gb	number	Block storage size in GiB	200
grafana_port	number	Grafana HTTP port	3000
allowed_grafana_ipv4	list	IPv4 CIDRs for Grafana access	["0.0.0.0/0"]
tags	list	Resource tags	["infra", "obs"]

Table C.5: observability variables

Appendix D

HTTP Route Reference

Method	Path	Description	Role
GET	/	Home / dashboard	auth
GET	/catalog	Product catalog	auth
GET	/catalog/{id}	Product detail	auth
GET	/orders	My orders list	auth
GET	/orders/new	New order form	auth
POST	/orders/new	Submit order	auth
GET	/orders/{id}	Order detail / status	auth
GET	/projects	Projects list	auth
GET	/infrastructure	Infrastructure overview	auth
POST	/infrastructure/{id}/decommission	Trigger decommission	auth
GET	/settings/profile	Profile / change password	auth
GET	/approvals	Approval queue	admin
POST	/approvals/{id}/approve	Approve order	admin
POST	/approvals/{id}/reject	Reject order	admin
GET	/audit	Audit log	admin
GET	/audit/export	Export audit log	admin
GET	/admin	Admin dashboard	shop_admin
GET	/admin/categories	Category list	shop_admin
GET	/admin/products	Product list	shop_admin
GET	/admin/products/new	New product form	shop_admin
GET	/admin/products/{id}	Edit product	shop_admin
GET	/admin/environments	Environment list	shop_admin
GET	/admin/sources	GitLab sources list	shop_admin
GET	/admin/costcenters	Cost center list	shop_admin
GET	/admin/users	User list	shop_admin
GET	/admin/currencies	Currency management	shop_admin
GET	/admin/parameters	Global parameters	shop_admin
GET	/admin/smtp	SMTP configuration	shop_admin
GET	/admin/ai-config	AI translation config	shop_admin
GET	/admin/branding	Shop branding	shop_admin
GET	/login	Login page	public
GET	/auth/callback	OIDC callback	public
POST	/logout	Log out	auth

Table D.1: HTTP route reference (selected)

Appendix E

Requirements Traceability Matrix

The following table maps each functional and non-functional requirement to the implementation location and the handbook section that describes it.

ID	Requirement	Implementation	Section
FA-01	User authentication (local + SSO)	internal/service/impl/user.go , internal/handler/auth.go	§8
FA-02	Role-based access control (admin, user, pm)	internal/middleware/auth.go , internal/model/user.go	§8
FA-03	Product catalog with categories	internal/handler/catalog.go	§13
FA-04	Configurable product parameters	internal/model/parameter.go , admin UI	§14.6
FA-05	Shopping-cart order flow	internal/handler/order.go	§13
FA-06	Order approval workflow	internal/handler/approval.go	§13
FA-07	GitLab webhook trigger	internal/service/impl/pipeline.go	§4.5
FA-08	Pipeline status polling	internal/polling/ , internal/service/impl/pipeline.go	§4.5
FA-09	Infrastructure overview	internal/handler/infrastructure.go	§15.2
FA-10	Decommission (destroy) flow	internal/handler/infrastructure.go	§13.5
FA-09.5–9.8	Cascade decommission on project/product/category delete	internal/service/impl/infrastructure.go	§13.5
FA-11	Audit log	internal/audit/ , internal/handler/audit.go	§9
FA-12	AI-powered product translation	internal/service/impl/translation.go	§7.4
FA-13	Multi-currency support	internal/service/impl/currency.go	§7.6
FA-14	Branding configuration	internal/handler/admin.go , internal/model/branding.go	§7.5
NFA-01.1	Docker deployment	Dockerfile , infra/docker-host/	§6.3
NFA-01.2	Kubernetes deployment	infra/helm/infra-webshop/ manifests	§6.4
NFA-01.3	Single binary (embed.FS)	cmd/server/main.go , <code>//go:embed</code> directives	§3
NFA-01.4	Database migrations at startup	cmd/migrate/main.go , internal/migrations/	§11.6
NFA-02.1	HTTPS enforcement	Nginx config (reverse proxy, TLS termination)	§6.3
NFA-02.2	Secure session cookies	internal/session/ , AES-GCM encrypted HttpOnly cookies	§3
NFA-02.3	bcrypt password hashing	internal/service/impl/user.go	§3
NFA-02.4	Role-enforced routes	internal/middleware/auth.go	§8
NFA-03	25-language i18n	internal/i18n/i18n.go	§A
NFA-04	Audit trail for all state changes	internal/audit/service.go	§9

Table E.1: Requirements traceability matrix

Appendix F

Compiling This Handbook

The handbook source is at [docs/handbook.tex](#). Compile with:

```
# From the repo root
make docs

# Or manually (run twice for ToC/references)
cd docs
pdflatex handbook.tex
pdflatex handbook.tex
```

Required LaTeX packages: lmodern, geometry, fancyhdr, xcolor, tikz, pgf, booktabs, longtable, tabularx, listings, tcolorbox, hyperref, enumitem, float, caption, graphicx, microtype, parskip, pdfscape, pifont.

All packages are available in TeX Live 2023+ and MiKTeX 23+.